

Sargantana Core Tile

High Level Architecture Specification

Description

The Sargantana Core Tile comprises: the Sargantana 64 bit RISC-V general purpose Central Processing Unit (CPU), the instruction cache, the High Performance Data Cache (HPDCache), the Memory Management Unit (MMU), and the Control and Status Registers (CSRs).

Revision History

Revision	Date	Author(s)	Description
0.1	18.12.2023	Víctor Soria	Created
0.2	21.12.2023	Víctor Soria	MMU Extension
0.3	8.2.2024	Narcís Rodas	RVV 1.0 Update
0.4	6.3.2024	Narcís Rodas	GMX CSRs
0.4.1	15.4.2024	Max Doblas	General formatting
0.5	16.4.2024	Narcís Rodas	SIMD new instructions
0.6	22.5.2024	Narcís Rodas	More SIMD instructions
0.7	24.5.2024	Arnau Bigas	HPM Section
0.8	28.6.2024	Arnau Bigas	Debug Mode Extension
0.9	4.7.2025	Narcís Rodas	B Extension
0.9.1	17.7.2025	Narcís Rodas	Removed GMX CSRs
0.9.2	22.7.2025	Narcís Rodas	Fixed memory pipeline section
1.0.0	28.7.2025	Narcís Rodas	Future Work & References sections
1.1.0	17.4.2026	Narcís Rodas	SIMD FPU, Hypervisor and Cache-block instructions
1.2.0	30.4.2026	Narcís Rodas	Reworked CSR section, updated supported extensions
1.3.0	4.6.2026	Narcís Rodas	Updated supported extensions

1 Description

Sargantana Core Tile wraps different RTL modules to implement a general purpose CPU following the IMAFDBH_Zicntr_Zicnd_Zicsr_Zihpm_Zfa_Zfhmin_Zvbb_Zvfhmin_Zicbom_Zicbop_Zicboz_Smcntrpmf_Sscfpmf_Sha_Sdext RISC-V standards and capable of booting Linux-based OS. The Sargantana Core Tile groups the following IPs:

- Sargantana: a 64-bit Single-Issue In-Order RISC-V processor [1].
- 16 KiB L1 Instruction Cache.
- 32 KiB L1 High Performance Data Cache (HPDCache) [2].
- The Memory Management Unit (MMU) composed by the Translation Lookaside Buffer (TLB) and the Page Table Walker (PTW).
- The Control and Status Registers (CSR).

Next sections describe the high-level microarchitecture of each IP.

2 Sargantana

Sargantana implements the RV64GBH_Zicntr_Zicnd_Zicsr_Zihpm_Zfa_Zfhmin_Zvbb_Zvfhmin_Zicbom_Zicbop_Zicboz_Smcntrpmf_Sscfpmf_Sha_Sdext Instruction Set Architecture. It features a 7-stage pipeline, a Floating Point Unit (FPU) and a 128-bit Single Instruction Multiple Data (SIMD) unit. Summarizing the main features of Sargantana:

- RV64GBH ISA with [V Extension subset v1.0](#)
- Supports machine mode, supervisor mode and user mode
- Pipeline: 7-stage single-issue in-order
- 64 Physical Registers
- 32 Entry Graduation list
- Bimodal Branch Predictor
- 8 Entry Load Queue & 8 Entry Store Buffer

2.1 Functional Description

2.1.1 Instruction Set Architecture (ISA)

The processor complies the RISC-V ISA [Volume 1 v2.1](#) & [Volume 2 v1.12](#) specifications. The base integer instruction set is [RV64I v2.1](#), and implements the following standard extensions:

- Integer Multiplication and Division ([M v2.0](#)).
- Atomic Instructions ([A v2.1](#)).
- Single-Precision Floating-Point ([F v2.2](#)).
- Double-Precision Floating-Point ([D v2.2](#)).
- Bit Manipulation ([B v1.0](#)). This extension includes instructions provided by the Zba, Zbb and Zbs extensions.
- Hypervisor (H v1.0).
- Base Counters and Timers (Zicntr v2.0).
- Integer Conditional Operations (Zicnd v1.0).
- Control and Status Register Instructions (Zicsr v2.0).
- Hardware Performance Counters (Zihpm v2.0).
- Additional Floating-Point Instructions (Zfa v1.0).
- Half-Precision Floating-Point Minimal (Zfhmin v1.0).
- Vector Basic Bit-manipulation (Zvbb v1.0).
- Vector Half-Precision Floating-Point Minimal (Zvfhmin v1.0).
- Base Cache Management Operation ISA (CMO v1.0).
- Count Overflow and Mode-Based Filtering (Sscofpmf v1.0).
- Cycle and Instret Privilege Mode Filtering (Smctrpmf v1.0).
- Supervisor-mode View of the State-enable (Ssstateen v1.0).
- Augmented Hypervisor (Sha, profile-defined).

Additionally, Sargantana supports a subset of the Vector Extension ([V v1.0](#)). It is only missing support for LMUL values greater than 1 and vector load/store segment instructions.

Sargantana supports RISC-V supervisor and user modes, providing three levels of [privilege](#):

- [Machine \(M\)](#). Highest privileges, applications that run in machine-mode are inherently trusted as they have low-level access.
- [Supervisor \(S\)](#). S-mode adds a number of additional CSRs and capabilities.
- [User \(U\)](#). Provides a mechanism to isolate application processes from each other and from trusted code running in M-mode.

2.1.2 Reset State

On the reset state, the PC is set to *reset_addr_i* (set to 0x0100 in the simulation testbenches), and all the flying instructions are flushed. The register file (except from the register x0, which is always 0) and the branch predictor stay in an undefined state. The instruction cache and data cache are invalidated on reset, losing all the containing modified values.

2.2 Instruction Pipeline

The Sargantana execution unit is a Single-Issue, In-Order pipeline with Out-of-Order write-back capabilities. The pipeline comprises 7 stages:

1. Instruction Fetch 1 (F1).
2. Instruction Fetch 2 (F2).
3. Instruction Decode (D).
4. Register Rename (R).
5. Read Register (RR).
6. Execution (E).
7. Register Write-Back (WB).

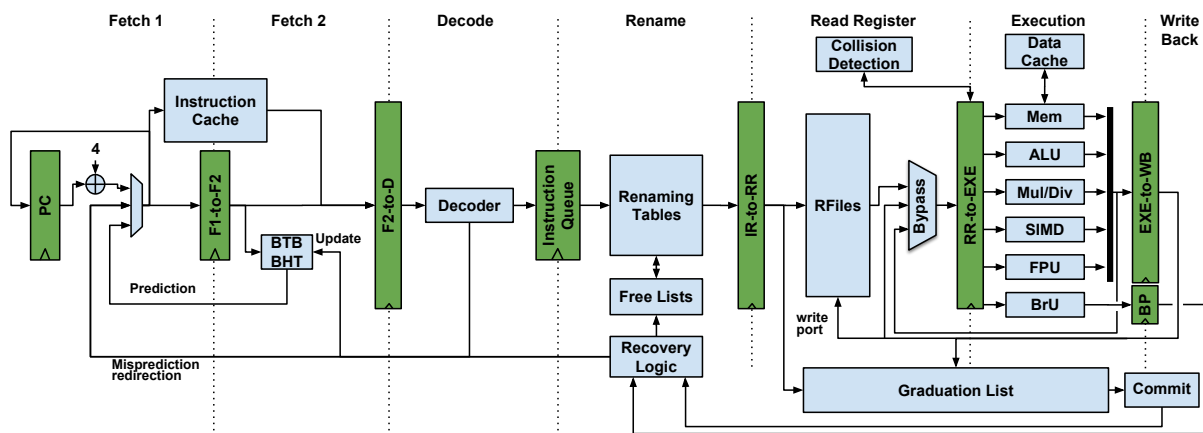


Figure 1: Sargantana 7-stage Pipeline Block Diagram.

The pipeline has a peak execution rate of one instruction per clock cycle, and is fully bypassed so that most instructions have a one-cycle result latency. The pipeline only interlocks on read-after-write hazards, so instructions may be scheduled to avoid stalls.

2.3 Instruction Fetch

The Sargantana Instruction Fetch Unit is responsible for keeping the pipeline fed with instructions from memory. The instruction fetch unit delivers up to 4 bytes of instructions per cycle. Fetches are always word-aligned. Otherwise, the fetch generates an instruction address misaligned trap. Trying to fetch from a non-executable or unimplemented address results in an instruction access trap.

The Sargantana instruction fetch unit contains predictive hardware to mitigate the performance impact of control hazards within the instruction stream. The instruction fetch unit is decoupled from the execution unit, so that correctly predicted control-flow events usually do not result in execution stalls.

- 128-entry Branch Target Buffer (BTB). Predicts the target of taken branches and direct jumps.
- 128-entry Branch History Table (BHT). Predicts the direction of conditional branches.
- 6-entry Return-Address Stack (RAS). Predicts the target of procedure returns.

The BTB is a 2-bit bimodal branch predictor. Direct jumps that miss in the BTB result in one-cycle fetch bubble. The RAS takes precedence over the BTB. If these structures' predictions disagree with the

BTB's prediction, a one-cycle fetch bubble results. Similar to direct jumps that miss in the BTB, the fetch bubble might not result in an execution stall.

Miss-predicted branches incur a five-cycle penalty, unless they depend on a multiplication, division or memory operation whose latency is larger than 5 cycles. Miss-predicted indirect jumps incur a five-cycle penalty.

2.4 Instruction Decode

The decoder takes a 32-bit RISC-V instruction as input and decodes it, generating all the different control wires for the different parts of the pipeline. The decoder also detects invalid instructions, recovers the execution flow in case of a JAL, and contains the return address stack predictor.

2.5 Register Renaming

The Renaming Register stage is responsible of mapping each of the virtual registers of the instruction to the physical registers. If the processed instruction generates some output data it allocates a new physical register to store its value. This stages uses two structures to perform these tasks: the rename table and the free list

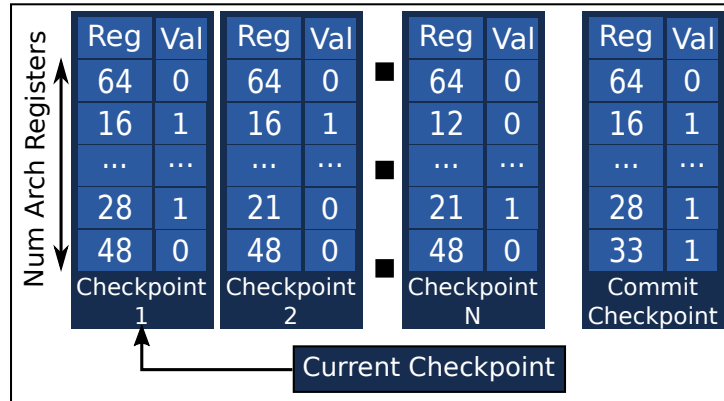


Figure 2: Block diagram of the renaming table.

The rename table is a structure that contains 32 entries that map each architectural register to one of the 64 physical registers that store the most updated value for the architectural register. Additionally, for each entry, there is a ready bit that indicates whether the value of the register is in the register file or must be caught on the fly using the bypass logic.

The structure has one table copy for each potential branch speculated in flight to recover a branch miss-prediction in one cycle. It also contains a copy of the renaming at the commit state for recovering an exception.

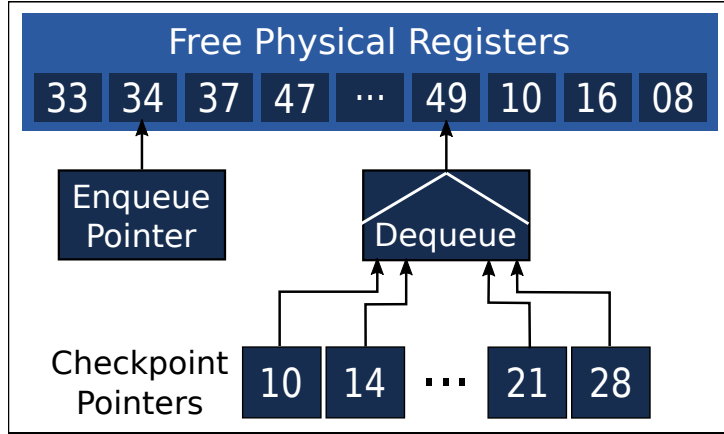


Figure 3: Block diagram of the free list.

The free list is a FIFO queue that stores all the registers that are not being used by any uncommitted instruction. It is a FIFO-like structure with an enqueue port for adding new free registers and a dequeue port to read physical destination registers to the rename stage. The dequeue port has several checkpoints for recovering a branch miss-prediction.

2.5.1 Recovery Logic

For recovering an exception or an interrupt, Sargantana keeps a copy of the entire rename table that is updated only at the commit stage. In the case of the free list, since it follows a FIFO order and a new register is only queued at commit, so, Sargantana sets the dequeue pointer to be just after the queued one.

In the case of branch miss-predictions, Sargantana can have several unresolved branches in-flight. To resolve this, it takes a checkpoint each time a branch instruction is renamed, and deletes the checkpoint or rolls-back to the previous state when the branch is resolved.

Therefore, the register rename table has N speculation checkpoints, and the *actual checkpoint* is a pointer to the version of the rename table that is used in this cycle. When a new branch instruction comes to the rename stage, the *actual checkpoint* is incremented, and the new checkpoint pointer is associated with the branch instruction. The previous checkpoint mapping is copied to the new checkpoint table. When a branch instruction is miss-predicted, Sargantana uses its checkpoint tag to recover the corresponding mapping.

The checkpointing mechanism in the free list consists of multiple versions of the dequeue pointer. When a branch miss-prediction occurs, Sargantana starts dequeuing again from that checkpoint.

2.6 Register File Read

Sargantana uses three different register files for the different types of operands:

- Integer. Has 2 ports for reading the 2 possible integer operands.
- Floating. Has 3 different reading ports, required by the floating point ISA extensions.
- SIMD. Has 4 different read ports.
 - 2 for the operand.
 - 1 for the old value of the destination register.
 - 1 for the mask, located on register $v0$.

These register files have 64 physical registers each to enable the disambiguation of the register renaming mechanism (there are 32 architectural registers).

2.6.1 Collision Detection

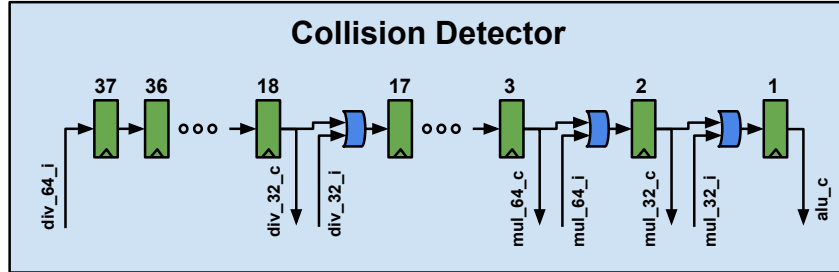


Figure 4: Block diagram of the collision detector.

Multiplication and Division Functional Units share a port because the latency on these modules is fixed and it is easy to detect collisions. The mechanism, consists of a shift register of size equal to the longest instruction latency.

When a new instruction is going to be issued, first, it checks that no other instruction is going to write-back in the same cycle. Then, if no collision is detected, it sets the bit in the position of the instruction latency minus one. This shift register copies the bit n to bit $n - 1$ at every cycle. If the module detects a collision, it delays the instruction for one cycle.

2.7 Execution Stage

Exceptions computes the actual outcome of instructions. There are several functional units to implement each of the instructions subsets. Most instructions are single cycle, the exceptions are the following:

Table 2: Instruction Latency Exceptions.

Instruction	Latency Cycles
LB, LH, LW, LD	3, assuming cache hit
MULW, MULH, MULHU, MULHSU	1
MUL	2
DIVW, DIVUW, REMW, REMUW	17
DIV, DIVU, REM, REMU	34

2.7.1 Memory Pipeline

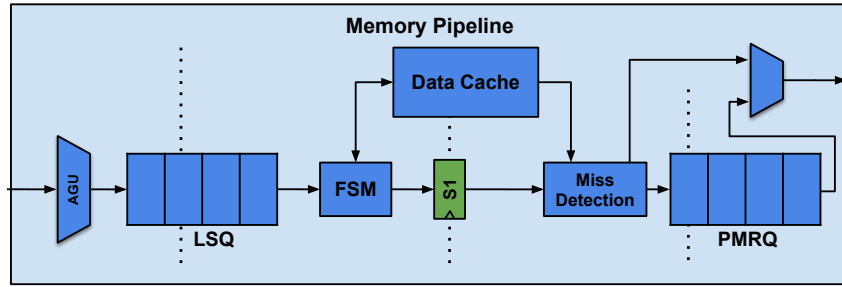


Figure 5: Block diagram of the memory pipeline. The instruction information of the memory access is pipelined alongside the cache. This information is merged with the cache response. In case of a cache miss, this information is stored on the PMRQ, waiting for future data response.

The memory Functional Unit is composed of a Load/Store Queue (LSQ), the L1 Data (L1D) cache and the Pending Request Memory Queue (PMRQ). The main feature of the L1D is its non-blocking design, which can sustain several requests in-flight thanks to a Miss Status Holding Registers (MSHR) table and a store buffer.

The access address is computed in the first stage, and the instruction is enqueued on the LSQ, a FIFO buffer. In the second stage, if the data cache can accept a new request (it can be blocked if the MSHR is full or if the instruction is a non-committed store), an instruction from the LSQ is dequeued and sent to the data cache. Since the latency on a data cache hit is two cycles, the instruction information is pipelined in parallel to the cache access. In the second cycle, if the cache answers with the request's response, the instruction is sent to write-back. Otherwise, the instruction is enqueued in the PMRQ to wait for its result. The PMRQ is a Content-Addressable Memory (CAM), so on every L1D cache answer, all entries are checked to wake up any waiting memory requests. The awoken instruction captures the data answer if necessary and advances to the write-back stage.

Load instructions can be issued speculatively to the data cache, but Stores and AMO must be executed in program order. If a Store or AMO arrives at the head of the LSQ, the control logic will wait until the same instruction arrives at the head of the graduation list to commit. This restriction delays the execution of younger ready loads that must wait until the execution of the blocking store due to the FIFO nature of the LSQ.

The LSQ contains another FIFO buffer named Store Buffer (SB). When a store arrives at the head of the LSQ buffer and it is not ready, instead of blocking the younger instruction, it is moved to the SB. When a load arrives at the head of the LSQ, first, it checks that the access address set and offset do not collide with any of the instructions stored in the SB.

2.7.2 Floating-Point Unit (FPU)

The Sargantana FPU [3] provides full hardware support for the [IEEE 754-2008](#) floating-point standard for 32-bit single-precision and 64-bit double-precision arithmetic. The FPU includes a fully pipelined fused-multiply-add unit and an iterative divide and square-root unit, magnitude comparators, and float-to-integer conversion units, all with full hardware support for subnormals and all IEEE default values.

Sargantana also supports the Zfhmin and Zfa extensions via execution units running parallel to the main FPU. To maximize hardware reuse, certain Zfa instructions are implemented in a post-processing stage after the CVFPU.

The FPU comes up disabled on reset. Initialize [FCSR](#) and set the FS configuration field of [MSTATUS](#) to 0x1 prior to executing any floating-point instructions.

2.7.3 Sargantana SIMD Unit

Sargantana’s SIMD unit is a module designed to implement the [RISC-V Vector extension ISA v1.0](#). Currently, it is missing instructions requiring an LMUL value greater than 1, which is an illegal configuration in our design. The length of the vector registers is fixed at 128 bits, and they can be configured to contain 8, 16, 32 or 64 bit-wide elements, using the [vsetvl](#) instruction. The module also offers support for masked operations, with the mask value always residing in vector register v0. It does not, however, support values of the vstart CSR different than 0. The Sargantana SIMD execution unit also supports RVV1.0 floating-point instructions by integrating the OpenHW’s group CVFPU[3] alongside new custom units. The CVFPU handles the majority of floating-point operations, but permutations, reductions and other missing features are handled separate dedicated units. Check Section 4

2.7.4 Vector memory operations

Sargantana supports the 3 addressing modes of the RVV specification for memory operations, unit-stride, strided and indexed (gather, scatter). It also supports 8, 16, 32 and 64-bit element width for all of these modes. The Vector Address Generation Unit is responsible of generating the needed requests (at a rate of 1 per cycle) to complete a vector memory instruction.

2.7.5 Graduation List

The graduation list is an ordering module to commit the instructions in order. On write-back, branches notify if the branch prediction was correct. In case of miss-prediction, the graduation list flushes all the instructions younger than the branch using the branch ID.

The graduation list implementation consists of a circular buffer with two pointers to keep track of the first and the last valid instructions. Each time an instruction is introduced on the graduation list, an instruction ID is returned, and the last valid instruction pointer is incremented. The instruction carries this ID until it finishes its execution. On write-back, the instruction sends its ID to the graduation list to mark the instruction as finished.

2.7.6 Commit

On every cycle, the commit stage checks if the two oldest instructions of the graduation list are completed. If only the oldest is completed, the commit stage dequeues the entry to process it. If both instructions are completed, the commit stage dequeues both entries to process it, giving priority in case of exception to the oldest one. If the instruction has an exception, the entire graduation list is flushed. A similar mechanism is used for branch miss-predictions.

3 ALU

The ALU is divided into three parts:

- Input sign- and zero-extension.
- Arithmetic-logic modules.
- Output sign and zero-extension and selection logic.

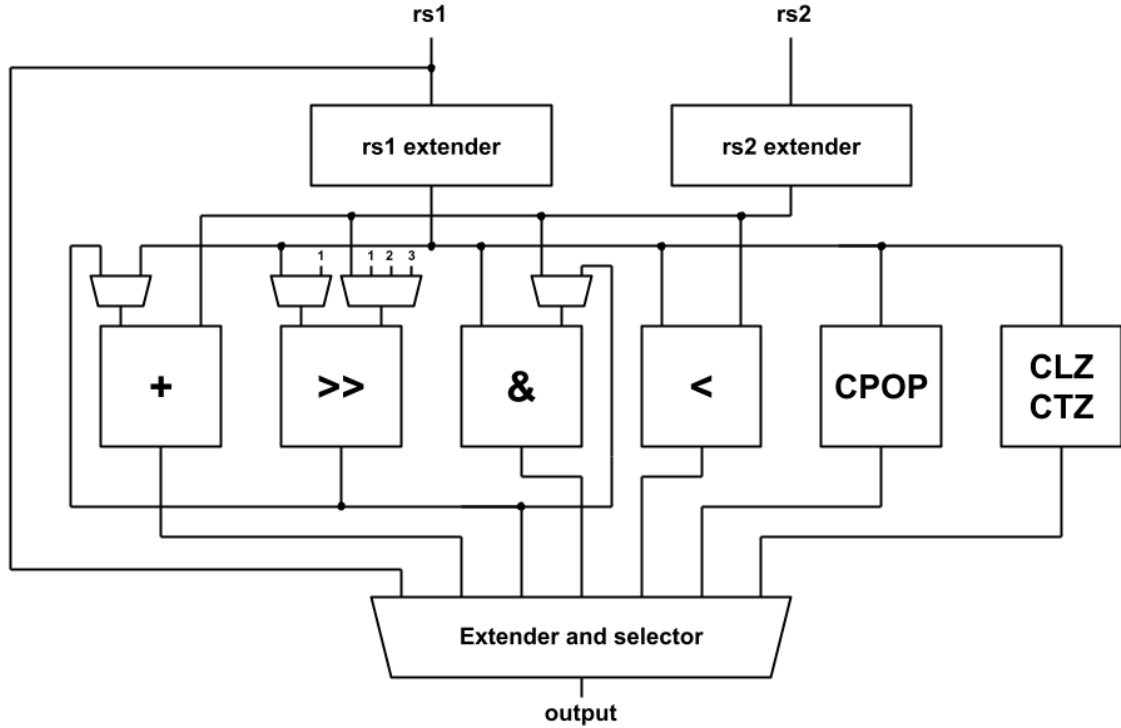


Figure 6: Block diagram of the ALU.

3.1 Arithmetic-logic modules

The ALU internally uses multiple arithmetic-logic modules, allowing it to perform all operations using a single module or multiple interconnected modules.

- **Addition module:** This module implements the negation and setting of the carry for the first adder in case of performing a subtraction and the addition itself.
- **Comparison module:** This module performs the signed and unsigned comparison between the two source registers and returns the result of this comparison, the maximum or the minimum, depending on which operation has been requested. It also implements the Integer Conditional Operations required by Zicond, which check the rs2 value against 0 and conditionally assign either rs1 or 0 to the destination register.
- **Shifting module:** This module shifts or rotates the data by using a funnel shifter, allowing for a single shifter that can do all the possible operations. The result of this module is also used to perform the shift and addition instructions in a single cycle by connecting its output to one of the inputs of the addition module, and for single-bit operations by connecting to one input of the logic module.
- **Logic module:** This module implements the logic operations (or, and, xor, or combine) between the input data. The result of this module is also used for the single-bit operations (set, clear, extract, and invert).
- **Count population module:** This module counts the number of set bits in the input data by using a tree reduction design.
- **Count leading zeros module:** This module counts the number of zeros at the least significant bits until reaching the first set bit. It can also perform the counting of trailing zeros by inverting

the order of the bits before performing the operation. The implementation of this module is based on the design done by Milenković et al. [4].

4 SIMD unit

4.1 Vectorial Floating Point Support

Vector floating point support is implemented using OpenHW’s group CVFPU, a state-of-the-art open-source, fully parametric and pipelineable unit. In fact, the Sargantana core already implements the CVFPU as the main scalar FPU.

Integration of the CVFPU

The CVFPU is instantiated as a submodule pointing to the main branch of the OpenHW’s group GitHub repository, and updates will be done periodically. To make the new CVFPU compliant with the core’s Flist format, there is a new script to recursively explore the CVFPU’s Flist format and update the core’s main Flist.

The interface of the updated CVFPU is listed and briefly described in table 3. These signals are driven by a decoder inside a wrapper unit named `vfpu_drac_wrapper` depending on the incoming instruction’s type. The most important ones are the operation selector signals, the operands’ array and the result bus, the source and destination format. As the mask is already managed inside the `simd_unit` module, the mask input is fixed to be always enabled for all the vector.

Apart from the previously mentioned signals, the CVFPU interface contemplates communication over val/rdy protocol and no fixed execution delays. This approach conflicts with the requisite of the *score-boarding* technique in use, as when the incoming instruction is inputted there is no knowledge about how many cycles it is going to take and thus structural hazards on the output cannot be detected nor avoided. Thus, a hybrid approach is taken: respecting the val/rdy protocol for the CVFPU and maintaining the use *score-boarding* for the rest of the execution modules.

I/O	signal type	signal name	description
input	logic	clk_i	Clock, synchronous, rising-edge triggered
input	logic	rst_ni	Asynchronous reset, active low
input	logic		
input	[NUM_OPERANDS-1:0] [WIDTH-1:0]	operands_i	Operands, henceforth referred to as op[i]
input	fpnew_pkg::roundmode_e	rnd_mode_i	Floating-point rounding mode
input	fpnew_pkg::operation_e	op_i	Operation select
input	logic	op_mod_i	Operation modifier
input	fpnew_pkg::fp_format_e	src_fmCVFPUt_i	Source FP format
input	fpnew_pkg::fp_format_e	dst_fmt_i	Destination FP format
input	fpnew_pkg::int_format_e	int_fmt_i	Integer format
input	logic	vectorial_op_i	Vectorial operation select
input	TagType	tag_i	Operation tag input
input	MaskType	simd_mask_i	Vector mask input for the status flags
input	logic	in_valid_i	Input data valid (see Handshake)
output	logic	in_ready_o	Input interface ready (see Handshake)
input	logic	flush_i	Synchronous pipeline reset
output	logic [WIDTH-1:0]	result_o	Result
output	fpnew_pkg::status_t	status_o	RISC-V floating-point status flags fflags
output	TagType	tag_o	Operation tag output
output	logic	out_valid_o	Output data valid (see Handshake)
input	logic	out_ready_i	Output interface ready (see Handshake)
output	logic	busy_o	FPU operation in flight
output	logic	early_valid_o	Output data valid in the next cycle

Table 3: Updated OpenHW’s group CVFPU module’s interface

This approach also requires the use of a *pending_queue* to store and re-order vector operations. This queue adapts the same module used for the scalar FPU. It is responsible for driving the val/rdy protocol on the CVFPU and assign each of the outputs to the correct instruction’s metadata to be committed. For simplicity, the implementation of this *pending_queue* consists of a FIFO queue that holds the metadata in arrival order and collects the CVFPU outputs. The idea here is that an instruction cannot be outputted if the result has not been received. Thus, the outputs will always leave in the order of arrival and will have to wait for previous uncompleted instructions.

If at any point, the CVFPU or the *pending_queue* are not able to receive the new incoming instruction, if the unit of the requested instruction is occupied or the FIFO is full, the front-end of the core is stalled and the core waits for the respective unit to be freed.

As a consequence of the hybrid approach, an arbiter must be used to avoid structural hazards on the single vectorial write-back port. For this purpose, a simple priority arbitration is done, giving always priority to the *score-board* bus and pushing the *pending_queue* ready signal to ‘low’ when collision is detected. To avoid the risk of stalling endlessly the CVFPU path, collision will also cause the stall of the front-end, creating space for the *pending_queue* output bus. In figure 7 a simplified diagram of the resulting *simd_unit* is displayed.

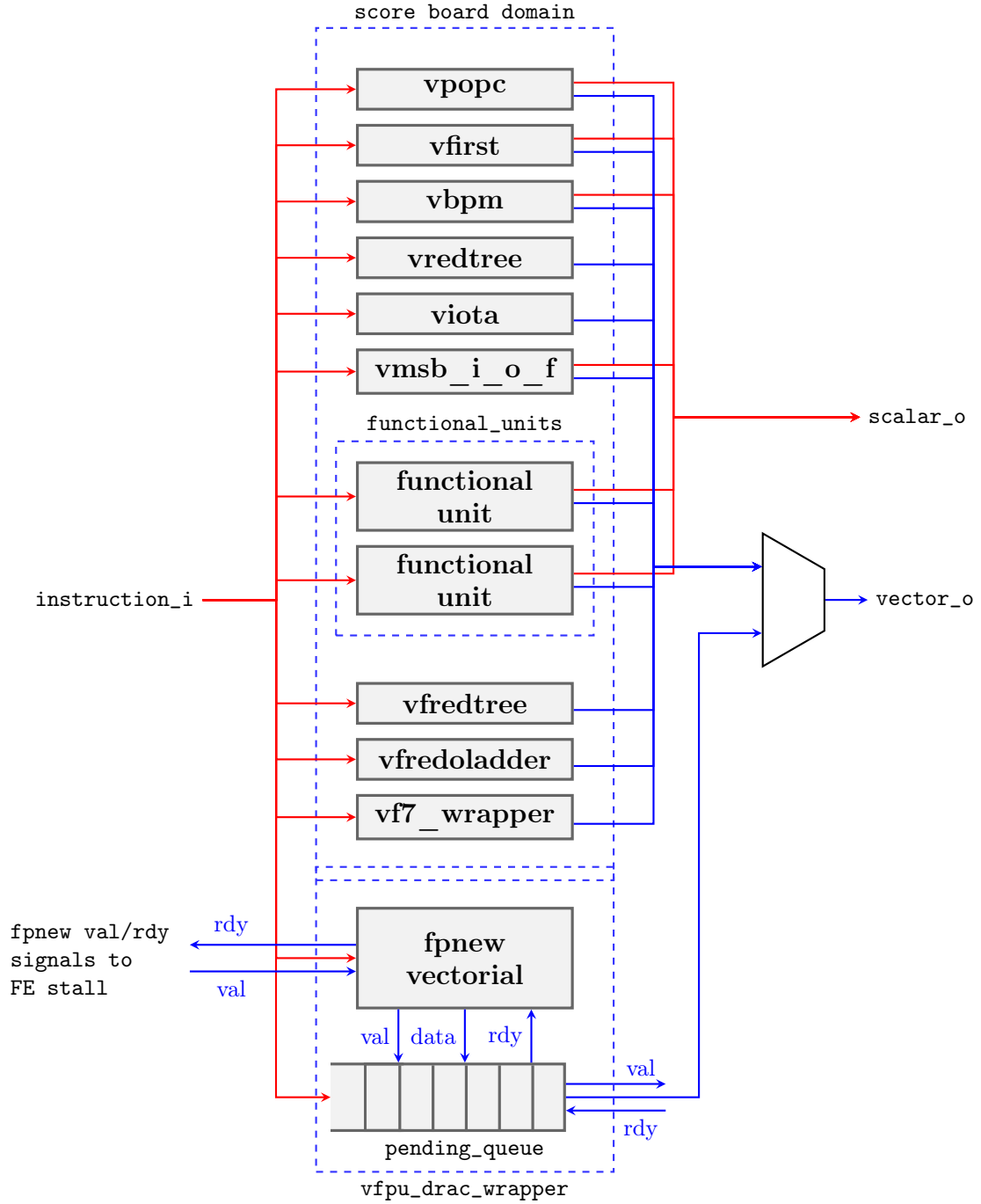


Figure 7: Simplified diagram of the new Integer+FP SIMD unit of the Sargantana core

4.1.1 Vectorial floating-point reductions

The implementation of floating-point vector reduction instructions has been divided across two new functional units instantiated directly at *simd_unit*. One module (*vfreditree*) performs unordered reductions including MAXMIN whereas the other one (*vfredoladder*) performs the 2 ordered instructions, viewed in table 4. Both modules are fully pipelined to fit the requirements of Sargantana’s SIMD’s present control logic. Both modules essentially perform the same reduction operations, but the ordered instructions are fixed to a specified operation order, see section 30.14.3.1, at the ratified RISC-V spec volume I.

instruction	module	description
Vector single-width floating-point reduction instructions		
vfredosum	<i>vfredoladder</i>	ordered sum reduction
vfredusum	<i>vfredtree</i>	unordered sum reduction
vfredmax	<i>vfredtree, comp_f32, comp_f64</i>	$\max(\text{vs1}[0], \text{vs2}[0], \dots \text{vs2}[\text{vl}-1])$
vfredmin	<i>vfredtree, comp_f32, comp_f64</i>	$\min(\text{vs1}[0], \text{vs2}[0], \dots \text{vs2}[\text{vl}-1])$
Vector widening floating-point reduction instructions		
vfwredosum.vs	<i>vfredoladder</i>	ordered widening sum reduction
vfwredusum.vs	<i>vfredtree</i>	unordered widening sum reduction

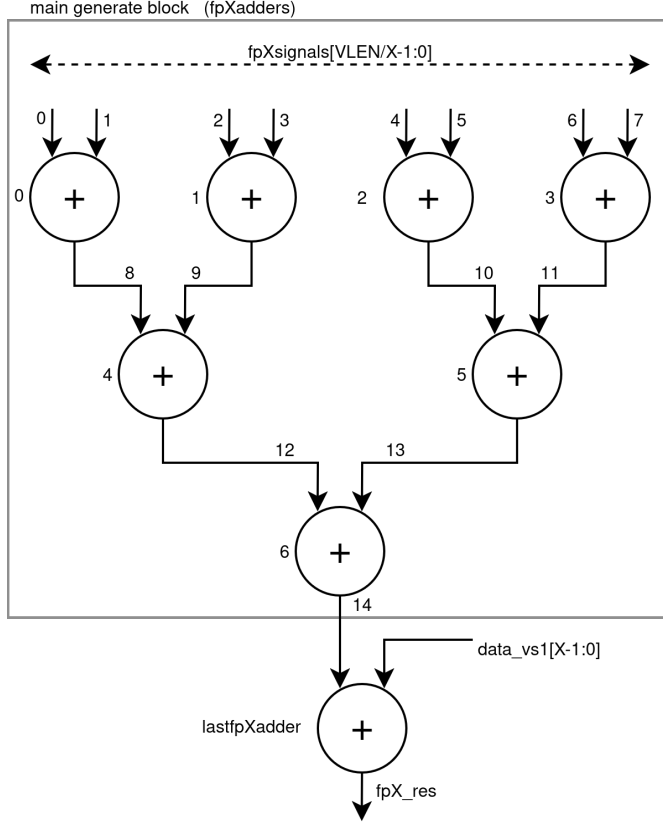
Table 4: Overview on the implemented vector floating-point reductions

***vfredtree* module**

This module is responsible for performing unordered vector floating-point reductions, including the *MAXMIN* reductions and widening form of the unordered *SUM* reduction. The implementation follows a tree structure to obtain a performance increase from the ordered $O(n)$ to an unordered $O(\log(n))$.

The unit wraps two reduction trees, one for each supported SEW: 32 bits and 64 bits. These instantiate respective *FPNEW* units, each of them responsible for adding pairs of 2 elements. These units are configured for FP32 and FP64 modes, respectively, as well as to only include the ADDMUL kernels.

The 2 trees share a unique output point of write back, taking advantage of the present SIMD scoreboarding technique of Sargantana’s SIMD pipeline. It ensures that 2 SIMD floating-point instructions will never end in the same cycle. The outputs are selected via the requested output instruction type and SEW. To be able to re-use the current scoreboarding technique, both reduction modules are fully pipelined accepting up to 1 instruction per cycle, which makes sense inside the Sargantana’s single-issue context. With simplicity in mind, the inner pipelines have kept constant delays for each SEW type, to be controlled only in the SIMD score-board structures.



Finally, the present modules have also been parametrized to accept, at synthesis time, different *vector legths* (VLEN). Regarding unordered vector reductions, each tree has been defined inside a main generate block iterating by the number of adders ($VLEN/X - 1$). The interconnections between adders have been described using the *fpXsignals* arrays of size $2 * (\frac{VLEN}{X} - 1) + 1$, where X is the SEW of the tree (either 32 or 64 bits for present Sargantana's version). The first $VLEN/X$ connections will be reserved to map there the first masked operands coming from the *data_vs2* input. The rest of the operands will be indexed in the connections array by $2i$ and $2i+1$ for sources and $\frac{VLEN}{32} + i$ for destinations, where i will be the correspondent iteration of the generate block. This will result in the structure shown in Figure 8.

Figure 8: Simplified signal diagram for *vfredtree* main generate block. $X \in \{32, 64\}$

In order to control the state of each stage in the reduction trees' pipelines and perform the correspondent operations, each tree has been accompanied by a register chain holding the instructions' *metadata*. This metadata shift register also serves the purpose of propagating down the pipeline the outgoing flags from each stage. The metadata structure fields are described below in Table 5.

type	name	description
<code>fpnew_pkg::status_t</code>	status	floating point flags, will accumulate through OR ladders the flags outgoing from each stage
<code>sew_t</code>	sew	SEW of the instruction
<code>instr_type_t</code>	instr_type	instruction type
<code>bus_simd_t</code>	data_vs1	data_vs1 of the input to be added after the reduction tree
<code>op_frm_fp_t</code>	frm	floating point rounding mode of the instruction

Table 5: Fields in the *vfredtree* metadata register chain

There is still some required data that cannot be allocated on the *metadata* register chain. The missing data is the required per each addition cell or iteration, which will be pipelined following the number of cycles each adder will take to perform each operation. Each *adder cell* is divided in two important cycles: the first, where the inputs are acquired and treated accordingly to be inputted to the adders, and the last, where outputs from the adder are processed, selected and sent to further stages of the tree. The required data in the latter cycle needs to be propagated from the first cycle through a set of smaller register chains

or shift registers. This is only for the two input operands of the stage and the validity signal for each of them propagating the input *vector mask*. Adding this changes, the resultant tree is the one displayed in Figure 9.

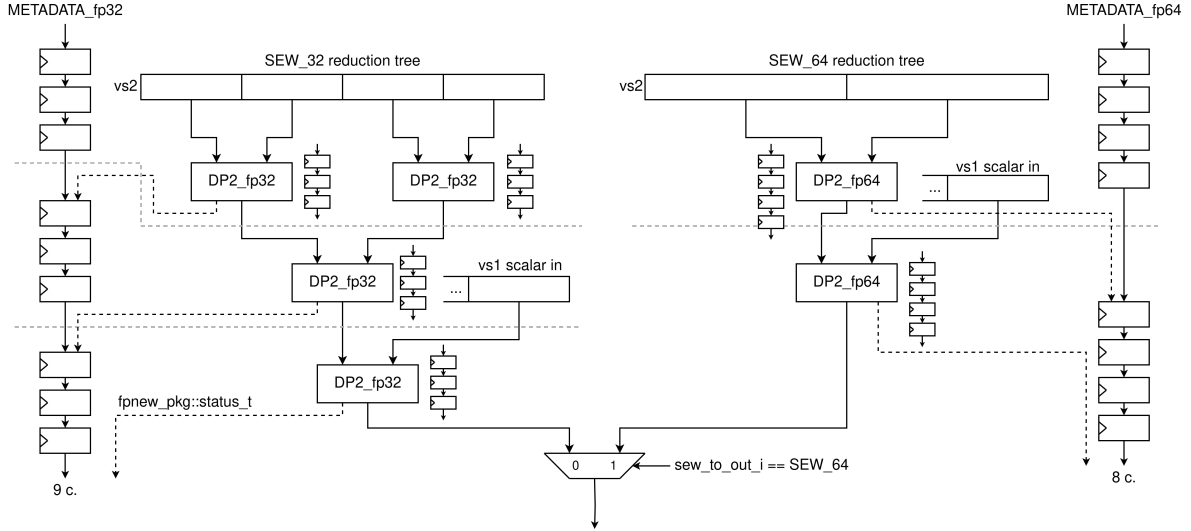


Figure 9: Simplified diagram of *vfredtree* module

As the MAXMIN reduction operations' specification does not impose a certain order, and these operations satisfy the commutative property, MAXMIN reductions are performed inside the *vfredtree* module in time $O(\log(n))$. To do so, each "adder cell" checks depending on the instruction type saved in the *metadata* structure and combinationally selects at the last cycle of operation the MAXIMUM, MINIMUM or canonical QNAN. Also, to perform those MAXIMUM or MINIMUM operations in compliance of the IEEE-754-2008 standard, *maxmin_fX* modules have been added. This cell's diagram is shown in Figure 10.

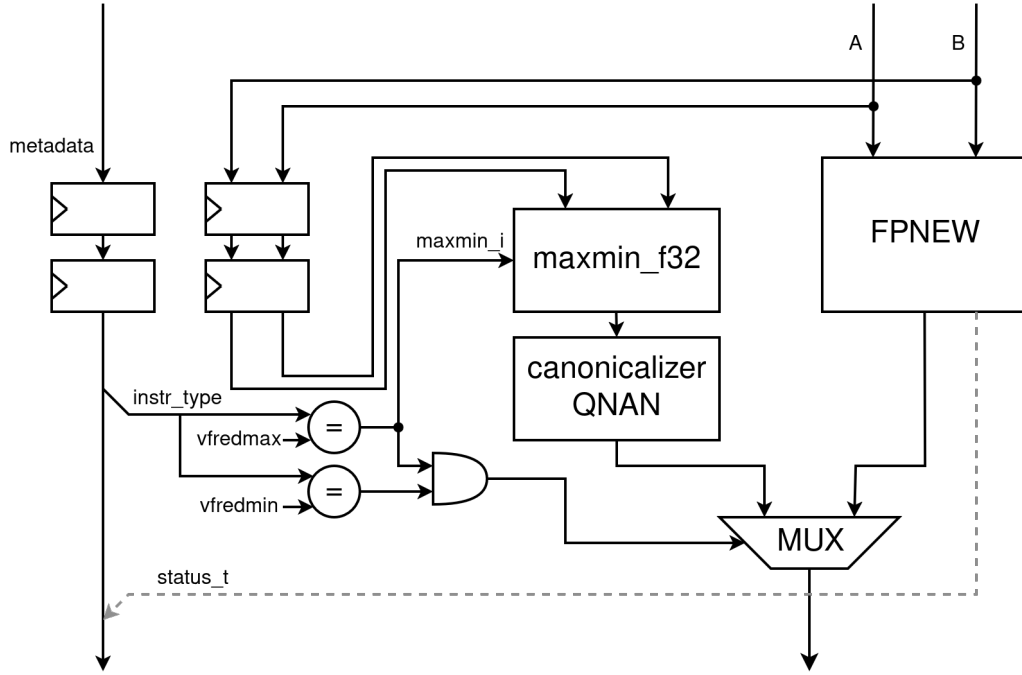


Figure 10: RTL diagram of the cell of *vfredtree* module

This module has undergone some structural changes regarding its support for widening unordered reductions. These are explained in further textbf 4.1.1.

vfredoladder module

The present ratified RISC-V spec differentiates two types of SUM reduction: the unordered and the ordered. This is due to the non-commutative behavior of the addition in floating-point arithmetics. Thus leaves freedom of implementation for a faster non-deterministic *unordered* design, but also forces to maintain the sequential and deterministic behavior in the ordered instruction.

The order given arbitrarily by the spec is the following:

$$vd[0] = (((vs1[0] + vs2[0]) + vs2[1]) + ...) + vs2[vl - 1]$$

As the implicit nature of the ordered reduction is different from the unordered, it was decided to design a new module for these operations. The *vfredoladder* module has been named after it's *ladder* geometry. It will perform operations following the specified order. It is fully pipelined, as the *vfredtree* module, and can accept up to one instruction per cycle. Also similarly to the unordered operations, there are 2 ladders instantiated inside the same module and a single write-back port is shared between them, selecting the output by the requested output SEW and instruction type.

In this case, the module has also been parametrized to generate the 32 and 64 bit ladders depending on the *vector lenght* (VLEN) of the Sargantana's design. The first adder is always treated differently, as it has to perform the addition of the scalar input ($vs1[X-1:0]$) and the first element of the vector register ($vs2$). The rest of vector operands are added one by one by a generate set of $VLEN/X - 1$ adders. To propagate the signals, a set of $2 * VLEN/X$ connections *fpXsignals* are used as shown in the simplified diagram in Figure 11.

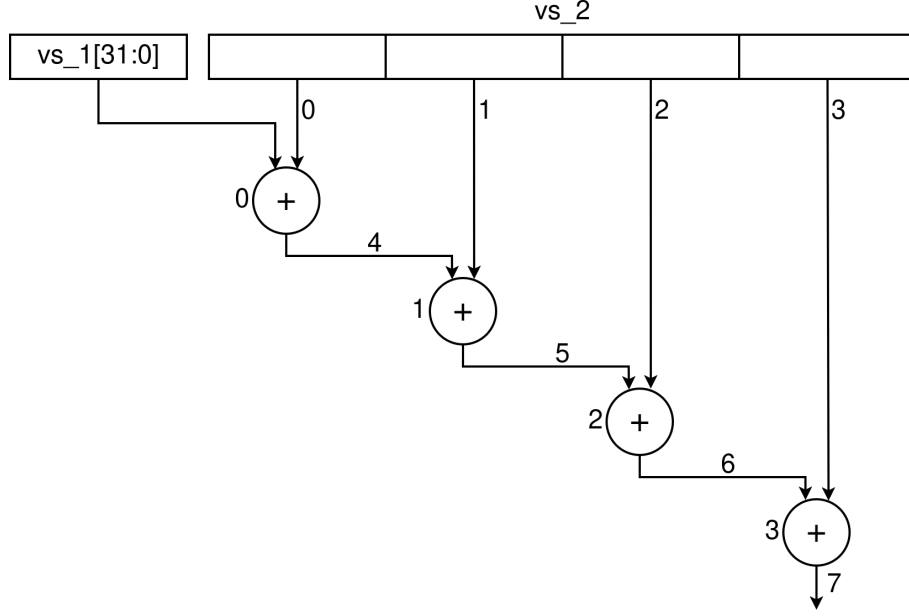


Figure 11: Simplified diagram of the *vfredoladder* module. Operands are shown as static vectors, but inside the pipeline’s context those should represent each cycle’s instructions. Numeration of signals represents indexes on fpXsignals logic arrays and adders’ indexes represent iterations of the generate block.

Similarly to *vfredtree* a *metadata* register chain is also necessary to propagate down the pipeline each instruction’s data. Luckily, each *metadata* stage is unique for an adder and thus is reusable to store the operands for each operation, apart from accumulating compute flags and propagating instruction-related information to further stages. This metadata vector has been defined as a matrix or a chain of sub-chains/stages: the first dimension is the iteration index of the generate block and corresponds to an adder module, the second is the delay each adder needs to compute its result. This choice simplifies even more the propagation of signals and further changes needed for providing widening reduction instructions’ support.

Widening instructions

In RISC-V ISA, there only exist 2 floating-point widening reductions, corresponding respectively to unordered and ordered additions. There are some important points to take into account regarding the widening operations:

- In widening operations, the scalar operand *vs1[63:0]* has to be already given in 64 bits width. Thus, it does not have to be widened.
- Contrarily to other widening operations in Sargantana, if the settled *vl* exceeds the FP64 *vector length*, the instruction needs to still be able to compute the reduction of the *VLEN/32* operands.

The proposed design for *vfusedusum* is to add an extra stage or layer in the inverse reduction trees to directly pass the existent tree its results. These results are selected through multiplexers at the first layer of the existent tree. The control of these multiplexers is whether there is a *vfusedusum* instruction coming out of the new level. This new layer is accompanied by a new chain of *widemetadatas*, which is also multiplexed at the initial assignments of the *metadata* structure. The resulting design for *vfredtree* is simplified in Figure 12.

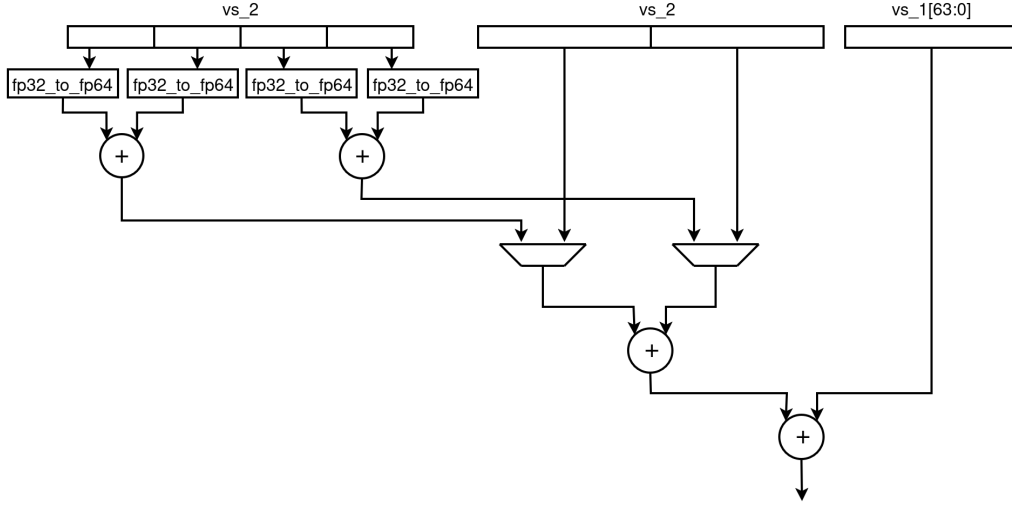


Figure 12: Simplified diagram of the *vfredtree* module with widening support

Some changes have also been done to the *vfredoladder* module. The idea is to maintain the functionality in case of not having a widening instruction, but at the same time, re-use the adders to avoid extra hardware. The proposal could be understood as overlapping a FP32 tree structure on top of the FP64 tree. More specifically, it consists on performing the first $VLEN/32 - VLEN/64$ operations before passing them to the already existent adders. To do so, the procedure mimics the existent structure for those extra operations: the first widened addition is done separately before entering a generate block with $VLEN/32 - VLEN/64 - 1$ adders. The FP64 structure is modified to operate both the first addition indistinctively by adding the scalar element when operating a regular reduction instruction, but at the same time, operate with the accumulated values and the next reduction element of the widened vector in the case of having a VWFREDOSUM instruction. In this case, the rest of the normal reduction operators are switched too.

The control requirements are very similar to the ones explained for the VWFREDUSUM widening instruction, as $VLEN/32 - VLEN/64$ extra *wide_metadata* is added.

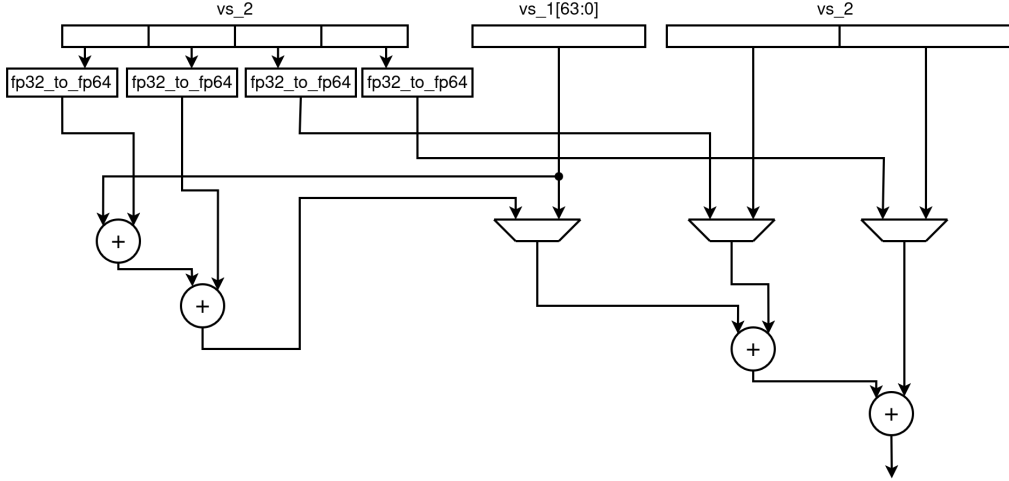


Figure 13: Simplified diagram of the *vfredoladder* module with widening support. Operands have been simplified for simplicity, but should be understood like the data propagated through the pipeline in the *metadata* structures.

This manner of re-using the hardware in both modules will strongly rely on the Sargantana’s SIMD score-boarding technique to avoid losing a valid incoming non-widening instruction during the cycles the widening instructions are injected as inputs.

4.2 VF7 approximation module

instruction	description
<code>vfrec7.v</code>	7 bits mantissa approximation of the reciprocal unary operation ($\frac{1}{x}$)
<code>vfrsqr7.v</code>	7 bits mantissa approximation of the reverse squareroot unary operation ($\frac{1}{\sqrt{x}}$)

Table 6: Overview on the implemented vector floating-point approximation Risc-V instructions

Another of the new auxiliary modules is *VF7*, which is the responsible for parallel computation of the RISC-V approximation instructions *VFRSQR7* and *VFREC7*, computations not performed by the current version of the *CVFPU*. These instructions estimate combinationally results to 7 valid precision bits for their respective operations.

The module is replicated 4 times inside its own wrapper inside the *simd_unit* module. The source operands are treated here, the input decoded and the inputs of *vf7_mf* driven. Also, the resulting status flags are reduced between units. As the rest of the SIMD operations, the *vf7_wrapper* is also parametric in *VLEN*.

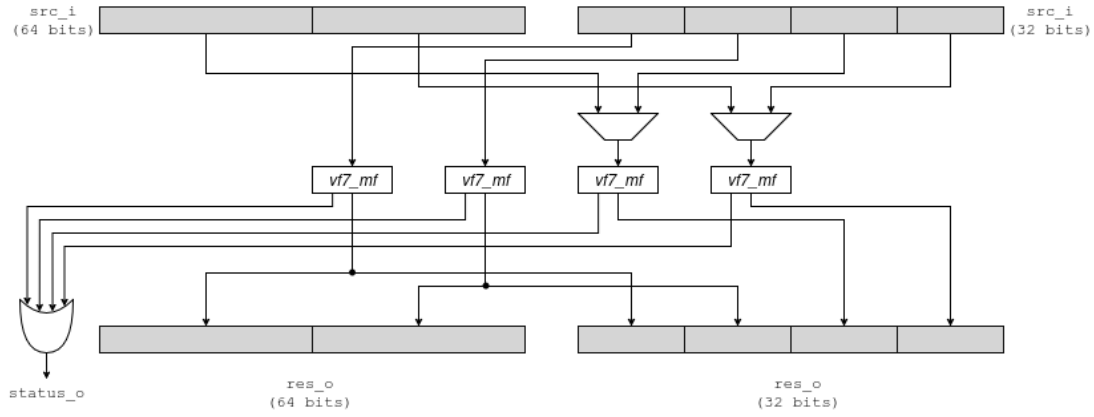


Figure 14: Simplified diagram of *vf7_wrapper*

type	name	description
logic	clk_i	system's clock
logic	rstn_i	system's reset
logic	valid_i	validity of the input instruction
drac_pkg::sew_t	sew_i	operational mode: SEW_32 or SEW_64
logic	operation_i	1'b0: VFREC7, 1'b1: VFRSQRT7
bus_simd_t	src_i	source operand
fpnew_pkg::roundmode_e	frm_i	roundmode, only for VFREC7
bus_simd_t	res_o	result output
logic	valid_o	validity of the output
fpnew_pkg::status_t	status_o	status output

Table 7: Description table of *vf7_wrapper* interface

The inner *vf7_mf* modules support two operation modes: VFREC7 and VFRSQRT7 respectively selectable through an input bit on their interface directly passed by the wrapper. Each of the operational modes allow for FP64 and FP32 operations, the latter properly NaN boxing. Both instructions take a similar approach to their respective computations, based on table look-up of given tables by the ratified *spec* and afterward normalization.

The computation is done purely combinatorially, thus both instructions will take only one cycle and be routed through the score-board domain vector output of the *simd_unit*. The RISC-V ratified specification does describe in depth the process to be done for each of the instructions. The *vf7_mf* module is distributed for each of the computation stages.

Summarizing, the combinational stages of the *vf7_mf* module are the following:

1. *Input decomposition* Decompose the input operand in sign, exponent and mantissa. Detect input cases: NaN, zer, inf...
2. *Normalization* Shift the mantissa left the number of leading zeros and normalize exponent when the number is *subnormal*. The case of *vfrec7* operation for direct infinite when values are big enough is also detected.
3. *Table look-up* The normalized mantissa, and LSB of exponent for *vf7sqrt7*, are used for look-up table indexing. The look-up tables remain declared as *localparam logic* types to avoid replication at synthesis.

4. *Result Computation* Edge cases described in the spec are treated and final result is put together and re-normalized on the corresponding output format in each case. Some output edge cases are also detected and returned accordingly: inf, zero and subnormality. Respective output flags are also returned.

In table 8 the interface of the inner module *vf7_mf* module is depicted. Note how most of the interface is shared with it's wrapper.

type	name	description
logic	clk_i	clock input
logic	rstn_i	reset input
logic	valid_i	when the input operand is to be treated (for VAL/RDY operations)
drac_pkg::sew_t	sew_i	operational sew (SEW_32 or SEW_64)
logic	operation_i	operational mode (0: VFREC7, 1: VFRSqrt7)
bus64_t	src_i	source input, top 64 bits ignored on SEW_32 operation
fpnew_pkg::roundmode_e	frm_i	input rounding mode, only considered for VFREC7 operation
bus64_t	res_o	result operand, SEW_32 is correctly NaN boxed on upper bits
logic	valid_o	result is valid (for VAL/RDY operations)
fpnew_pkg::status_t	status_o	result FP status flags

Table 8: Description table of *vf7_mf* interface

4.2.1 Permutations

The permutation instructions follow the same design as the integer ones, with the new need to control the *NaN* values and canonicalize them and also be sure that the elements are correctly *NaN*-boxed for the cases that $FLEN > SEW$.

This control has been added to the replication part of the SIMD unit for the rs1 register to generate a virtual vector register with this data.

By having this case controlled, the slides are directly executed with the same hardware as the integer one, performing a shifting of the elements of the vector register. And also for the moving of the data from a scalar floating-point register to the least significant element of the vector register.

For the instruction that moves the least significant element of a vector register to a scalar floating-point register, a new output port has been added to the SIMD unit that follows the format of a floating-point instruction. The data from this output port has been then connected to the execution stage module to a multiplexer that selects it or the output from the scalar FPU.

To avoid collisions between results from the SIMD unit and the FPU a new condition has been added to the write-back stalling signal of the FPU. If the current instruction entering the SIMD unit writes to the scalar floating-point registers and is a valid SIMD instruction, the FPU does not extract an instruction on this cycle.

4.3 Zvfhmin module

The Zvfhmin module is responsible for implementing the narrowing and widening conversions between floating point numbers when *SEW* is 16.

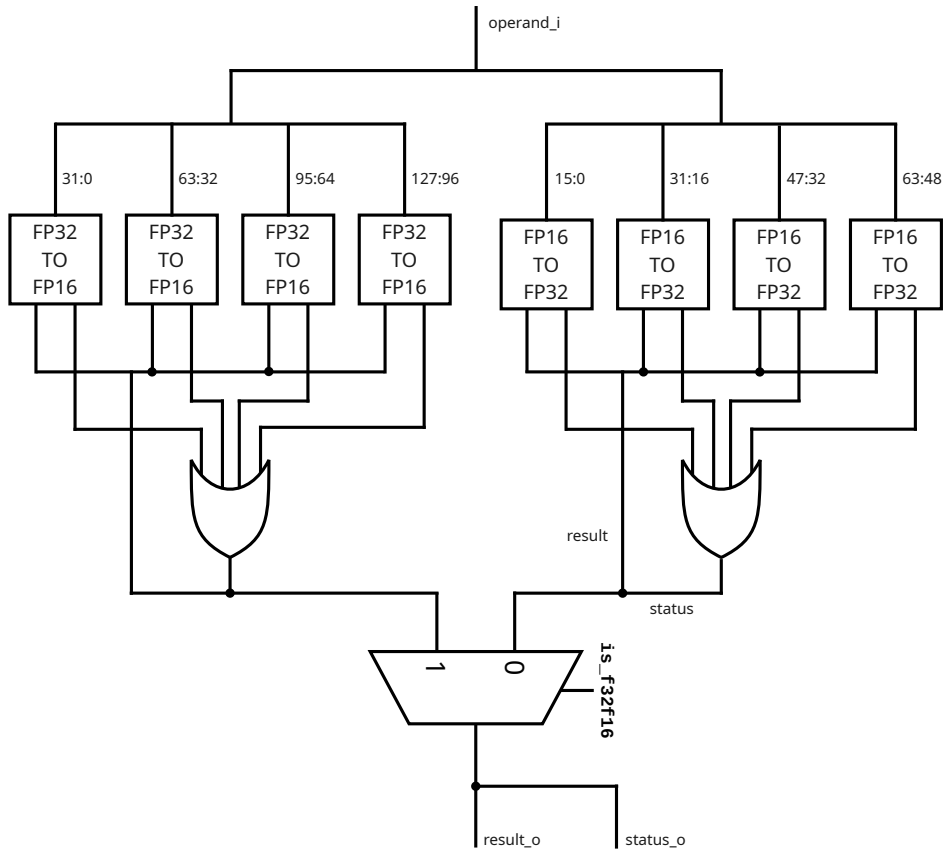


Figure 15: Diagram of the Zvfhmin module inside the SIMD unit.

This module implements two arrays of $VLEN/32$ widening and narrowing converters, from FP32 to FP16 and viceversa. After the conversion modules the partial results are packed into a $VLEN$ or $VLEN/2$ depending on if it is a widening or narrowing operation and the status flags are combined doing a simple OR operation. Then the results and statuses are multiplexed according to the type of input operation.

5 Cache System

5.1 L1 Instruction Cache

The L1 instruction cache is a 4-way set associative cache. The access latency of the cache is one cycle. It is virtually-indexed, physically-tagged with a line size of 32 bytes and is read/write-allocate with a random replacement policy. A cache line fill triggers a burst access outside of the core, starting with the first address of the cache line in 128-bit blocks. There are no write-backs to memory from the instruction cache and it is kept coherent with the memory system.

Out of reset, all blocks of the instruction cache are invalidated.

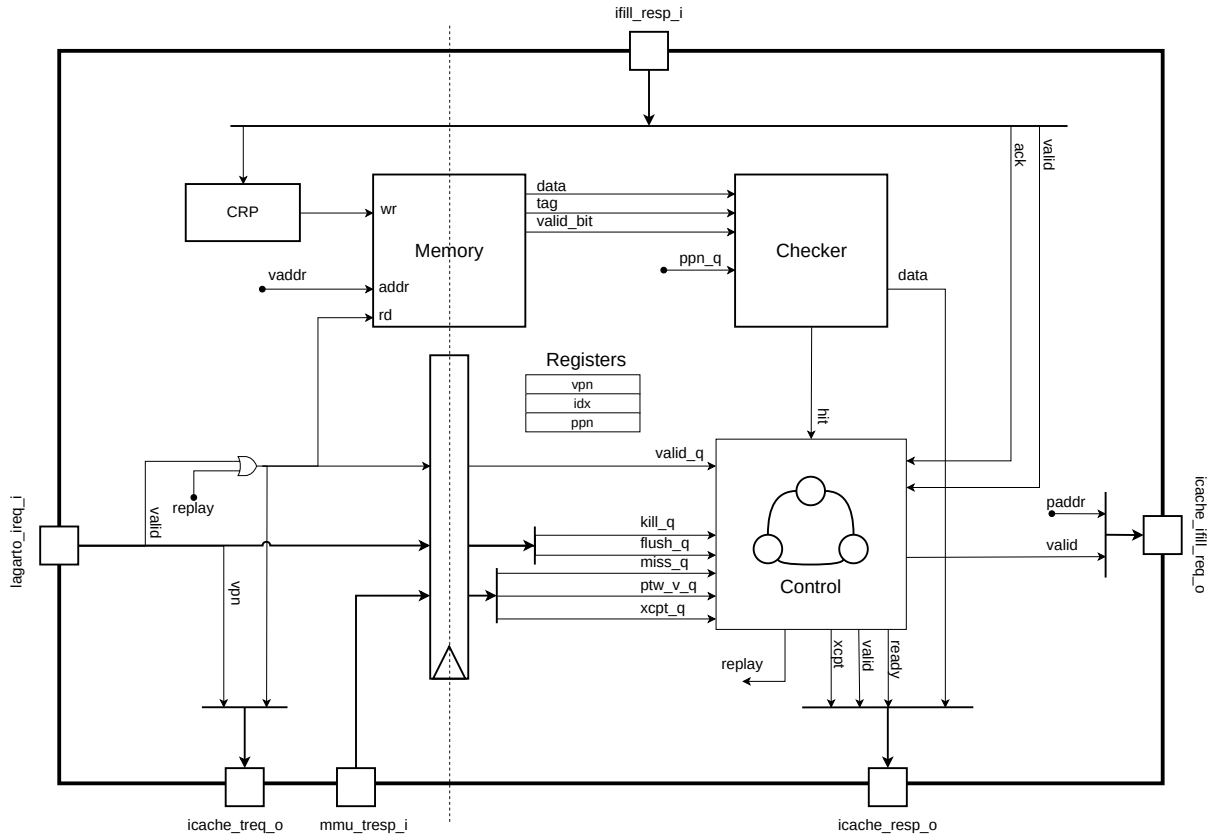


Figure 16: Instruction Cache Block Diagram.

5.1.1 Coherence with the L2 Cache

The L1 instruction cache is inclusive with the L2 Cache. When a block of instruction memory is allocated to the L1 cache, it is also allocated to the L2 cache. When a block is evicted from L1, it might still reside in the L2, which will reduce access time the next time the block is fetched.

If a hart modifies instruction memory (for example, self-modifying code) then a [FENCE.I Instruction](#) is required to synchronize the instruction and data streams. To maintain L1 instruction cache coherency the L2 might send snoop messages to invalidate a specific cache line.

5.2 L1 High Performance Data Cache

The High-Performance, Multi-Issue, Out-of-Order, L1 Dcache (HPDcache) [2] is the responsible for serving data accesses issued by a RISC-V core. The HPDC is 4-way set associative that stores 32Kib of data.

It is physically-indexed, physically-tagged with a line size of 64 bytes and is read-allocate/write-through with a Pseudo-LRU replacement policy. Figure 17 depicts a global view of the HPDcache. On the upper part of the cache there is the interface from/to requesters. On the bottom part there is the interface from/to the memory.

The first stage (stage 0) of the pipeline arbitrates between requests from the miss handler (refill), Replay Table (RTAB), and requesters; the second stage (stage 1) responds to loads (in case of hit) and to stores; the third stage (stage 2) is only used by loads in case of miss. In this last stage, the cache allocates a new entry in the Miss Status Holding Registers (MSHR).

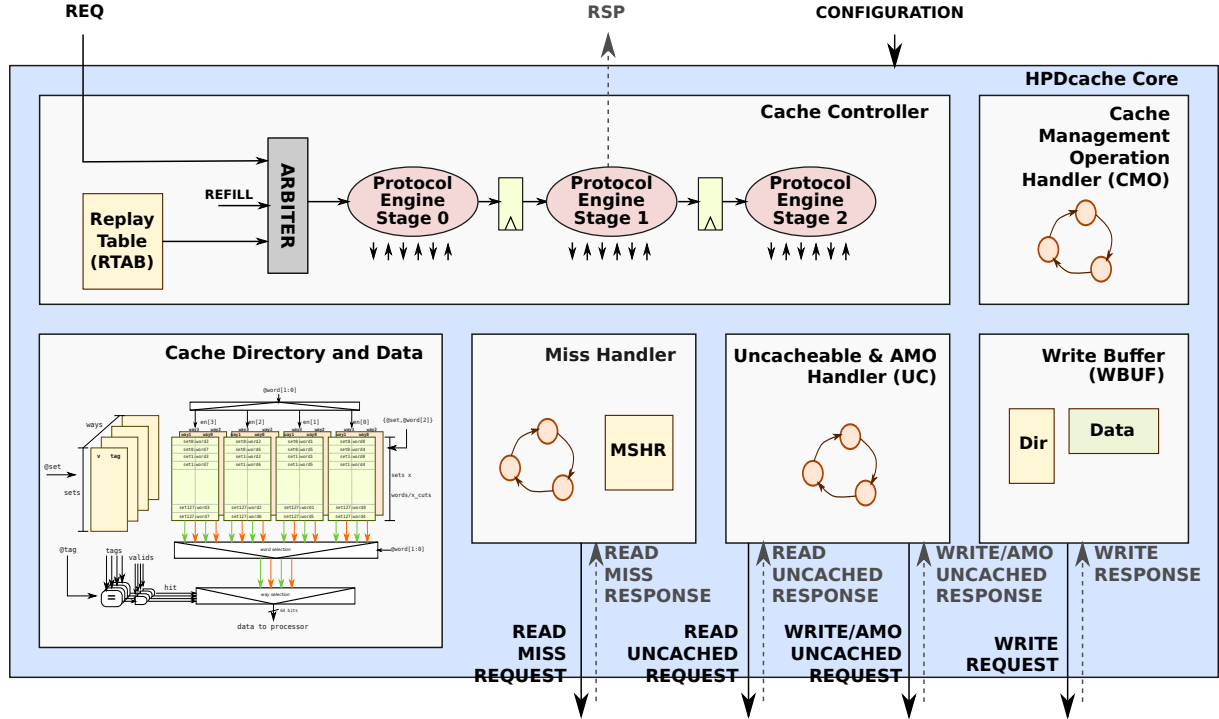


Figure 17: HPDC Block Diagram.

On a cache hit, the access latency is two cycles. Stores are stored in the Write Buffer (WBUF), which can coalesce multiple write request before forwarding the write operation to the L2. The data cache supports up to 64 outstanding line fills through its MSHRs. Data accesses are classified as non-cacheable if they are inside the I/O Address Space.

5.2.1 Cache Maintenance Operations (CMO)

The data cache supports CMOs, which are special requests from requesters that address the cache itself, and not the memory nor a peripheral. These operations allow to either invalidate designated cachelines in the cache, or produce explicit memory read and write fences.

5.2.2 L1 High Performance Data Cache Coherency

The HPDcache does not implement a hardware cache-coherency protocol, but it does support invalidations by address. Therefore, coherency can be enforced by configuring L2 cache to be inclusive. When a block of data is allocated to the L1 cache, it is also allocated to the L2 cache. As the cache implements a write-through policy, there is no memory-obsolescence problem. This is because all writes are forwarded to the memory.

5.2.3 Atomic Memory Operations

The CPUs support the RISC-V standard [Atomic Extension](#). The load-reserved (LR) and store-conditional (SC) instructions are special atomic instructions that are only supported in data cacheable regions.

6 Memory Management Unit (MMU)

Sargantana has support for virtual memory through the use of a MMU composed by a Translate Lookaside Buffer (TLB) and a Page Table Walker (PWT). The MMU supports the [Bare](#) and [Sv39](#) modes. The Sv39 implementation in both CPUs provides a 39-bit virtual address space using up to 56-bits of physical address space. Supported page sizes include 4 KiB, 2 MiB, and 1 GiB megapages.

6.1 Translation Lookaside Buffer (TLB)

The Translation Lookaside Buffer (TLB) serves as a cache for virtual-to-physical address translations, a key function in managing virtual memory. By storing recently used mappings, the TLB accelerates the translation process, minimizing the time required to access data in off-chip. TLB hits, where the translation is found in the TLB, result in faster access to physical addresses. In the event of a TLB miss, requiring a full translation, the result is stored for future use, maintaining efficiency.

In addition, this TLB also checks that the access rights to a page, raising a page fault in the case of a memory access violating them.

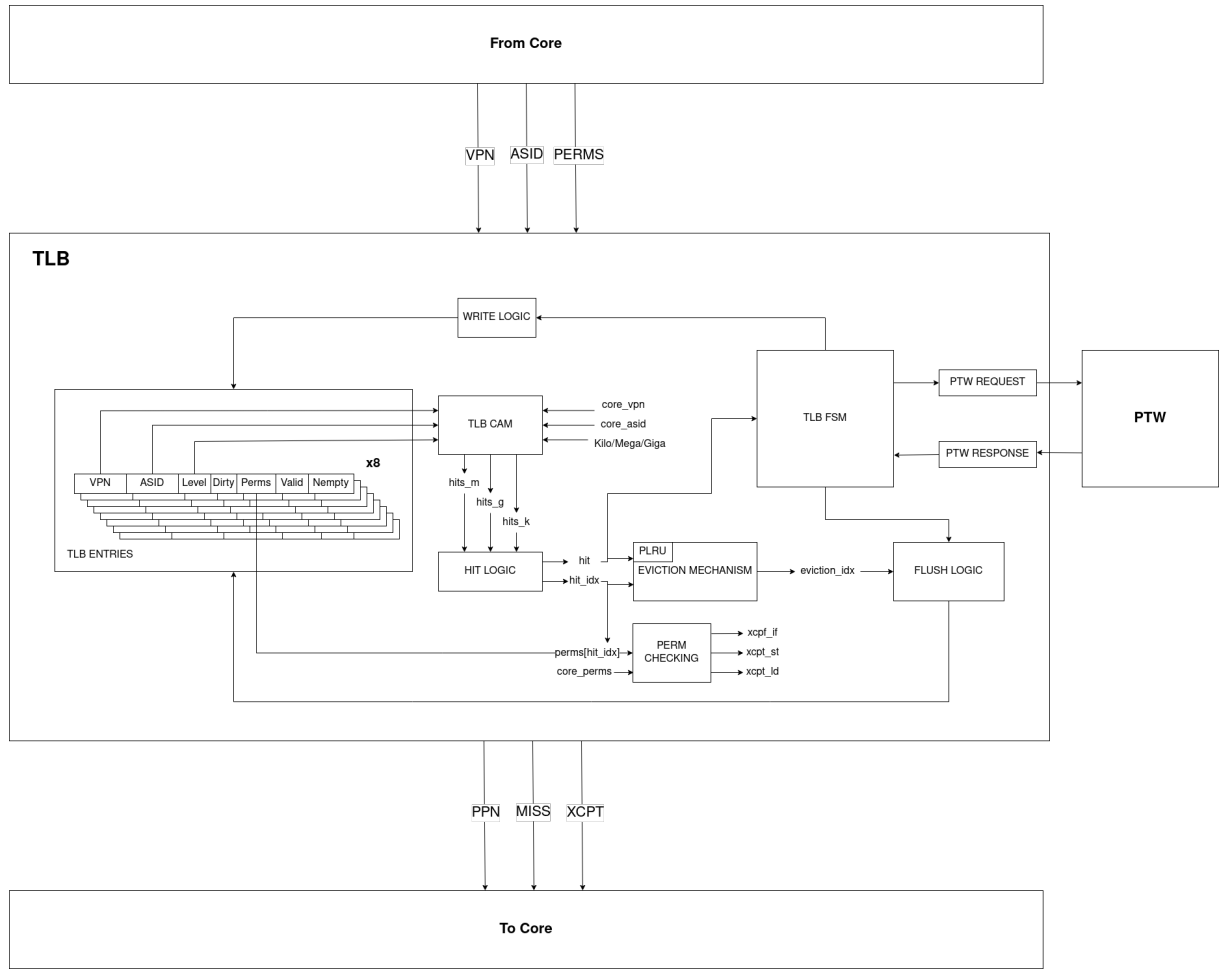


Figure 18: Transaction Lookaside Buffer Block Diagram.

In the case of a translation request not being stored in the TLB, we have a TLB miss, and it is requested to the page-table walker (PTW). The TLB blocks until the response is obtained, and the requested information should be kept stable until this response arrives. When the page-table walkthrough finalizes, the page-table entry (PTE) is stored in the TLB, using a pseudo-least recently used (PLRU) replacement algorithm. In the next cycle, the request hits and after checking the access permissions, it is sent to the requestor. The flush logic and eviction mechanisms ensure that empty and/or invalid PTEs that could be stored in the TLB are prioritized as the next slot to be written. The process of requesting a PTE to the PTW is implemented with a finite-state machine.

6.2 Page Table Walker (PTW)

A Page Table Walker (PTW) is responsible for translating virtual addresses generated by the CPU into physical addresses to access the main memory hierarchy system.

The Page Table Walker performs the translation by following a hierarchical structure called a page table. The page table organizes the mapping between virtual addresses and physical addresses. When the CPU core generates a virtual address, the Page Table Walker traverses the page table to locate the corresponding entry for that address. It follows a multi-level lookup process, where each level of the page table provides more specific details about the physical address corresponding to the virtual address.

The functionality of a Page Table Walker involves the following steps:

1. **Address Translation:** When the CPU core generates a virtual address, and after a TLB miss, the Page Table Walker starts the translation process by breaking down the virtual address into multiple parts (such as page number and offset).
2. **Page Table Lookup:** Using the hierarchical page table structure, the walker starts from the root of the page table (stored in satp register) and navigates through multiple levels, accessing intermediate page table entries (a.k.a PTE Table) to find the final mapping for the last page table entry with the virtual address (a.k.a PTE Leaf).
3. **The Page Table Entry Leaf (PTE Leaf):** The walker retrieves the corresponding PTE Leaf containing information like the physical page number (PPN), permissions bits (Read, Write, eXecute), status bits (Dirty, Access, Global mapping), and privilege status bit (User).
4. **Memory Access:** Finally, the walker provides the PTE Leaf with the actual physical address to the correspondent TLB, allowing the CPU to access the required memory location in dCache/Main Memory.

Our current PTW design supports SV39 (Virtual addresses of 39 bits), with 3 Levels of hierarchy. So, following the RISC-V privileged specs for SV39, the VPN has a size of 27 bits, and the PPN has a size of 44 bits. At the same time, the PTW supports Giga (1GB), Mega (2MB), and Kilo (4KB) pages. PTE Leafs received at Level 0 gives a Giga-page, at Level 1 a Mega-page, and at Level 2 a Kilo-Page.

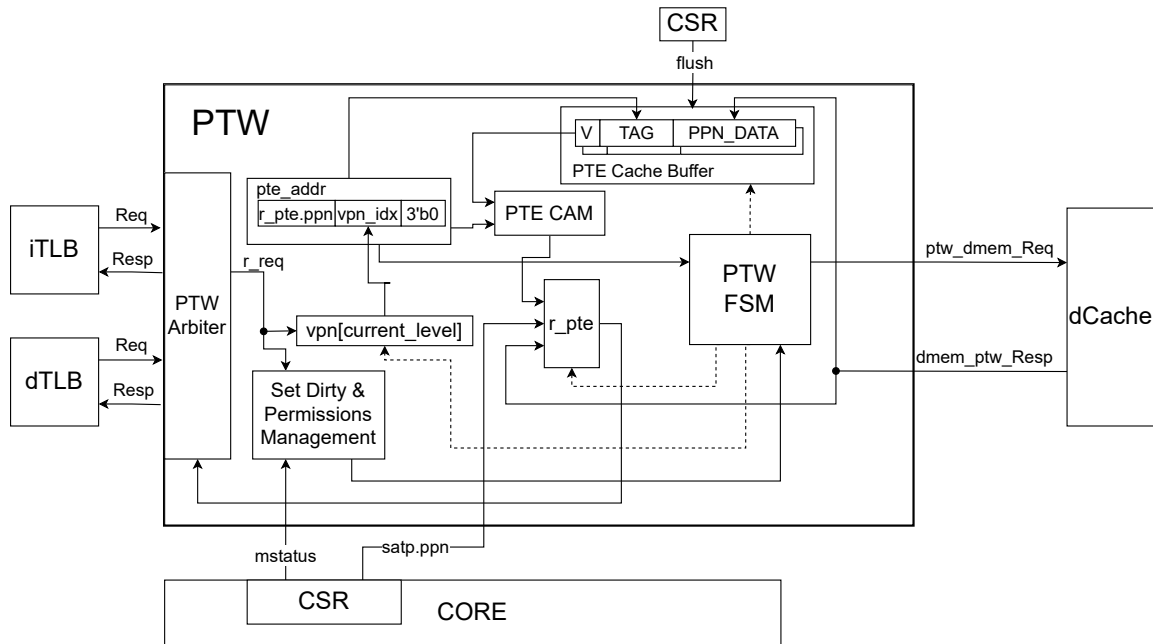


Figure 19: Page Table Walker Block Diagram.

The PTW includes an arbiter responsible for selecting a request when both the iTLB and dTLB send requests simultaneously. Priority is always given to dTLB requests over iTLB requests.

The PTW receives a Virtual Page Number (VPN) from the TLB, which is segmented into three parts, each corresponding to one of the three hierarchy levels.

The formation of the address for each Page Table Entry (PTE) at every level (`pte_addr`) is as follows:

1. For the initial level (L0), the PPN (Physical Page Number) field from the satp register sets the root of the main page table, with VPN[0] serving as an offset. Subsequently, the address is aligned to 8

bits before being sent to memory. The resulting address fetches the PTE of the subsequent level, stored in `r_pte`. If this PTE is a leaf, we stop the walk and we obtain a Giga-Page.

2. Upon reaching L1, and having a PTE Table, a request is made for the next level by combining the PPN of the current PTE (stored in `r_pte`) with `VPN[1]`. If the giving PTE is a leaf, we stop the walk and we obtain a Mega-Page.
3. The final PTE level (L2) must be a leaf node, representing a Kilo-Page. If it isn't, the PTW triggers an error signal to the TLB.

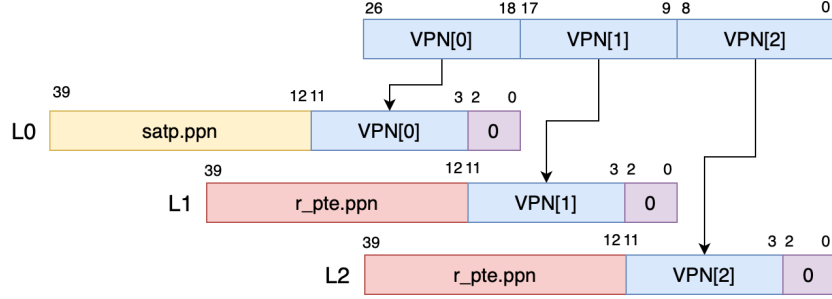


Figure 20: Page Table Entry Address Formation.

Once getting the PTE leaf, if all the privilege permission passes (comparing `mstatus` SUM and MXR bits with the request operation, fetch or store, and PTE bits), the PTE leaf proceeds to the Set Dirty Management. If the A bit (access) is not set, or the request is a store and the D bit (dirty) is not set, the FSM proceeds to write to memory setting the D and A bits. Once all this management logic concludes, the PTE, containing the corresponding Physical Page Number (PPN), is forwarded to the respective TLB.

In addition, to optimize the walking steps, the PTW contains a PTE Cache buffer. This buffer retains each received PTE Table from memory, using a pseudoLRU replacement strategy. Before initiating a request for a PTE from memory, the PTW checks this buffer cache using a Content-Addressable Memory (CAM). In the case where the TAG comparison registers a hit, the PTW can swiftly retrieve the PTE Table's PPN without the need for a memory request.

7 Hypervisor Support extension (H)

Sargantana implements the RISC-V Hypervisor extension (H) v1.0, which enables the core to run multiple guest operating systems atop a type-1 or type-2 hypervisor. The extension introduces a new virtualization mode (V-mode), two new privilege levels, two-stage address translation, a set of hypervisor and virtual-supervisor CSRs, and dedicated memory-access instructions.

7.1 Privilege Modes

The H extension adds a [virtualization mode](#) (`v_mode`) indicating whether the hart is currently executing in a guest mode. Combining that with the existing three privilege-level of Sargantana, yields five effective privilege levels:

Table 9: Privilege modes with the hypervisor extension.

v_mode	Nominal Privilege	Abbreviation	Name
0	U	U-Mode	User mode
0	S	HS-Mode	Hypervisor-extended supervisor mode
0	M	M-Mode	Machine mode
1	U	VU-Mode	Virtual user mode
1	S	VS-Mode	Virtual supervisor mode

- **Machine mode (M)**: unchanged, always non-virtualized.
- **Hypervisor-Supervisor mode (HS)**: supervisor privilege with `v_mode = 0`; used by the hypervisor itself.
- **Virtual-Supervisor mode (VS)**: supervisor privilege with `v_mode = 1`; a guest OS kernel would runs here.
- **Hypervisor-User mode (HU) or User mode (U)**: user privilege with `v_mode = 0`.
- **Virtual-User mode (VU)**: user privilege with `v_mode = 1`; a guest user processes would run here.

The virtualization mode also indicates whether two-stage address translation is active (`v_mode = 1`) or inactive (`v_mode = 0`).

7.2 Two-Stage Address Translation

When the virtualization mode is active, the Memory Management Unit (MMU) switches into a Two-step address translation scheme ([two-stage address translation](#)). In this context the MMU translates virtual addresses from virtual environments to physical ones.

Figure 21 Shows the two stages used to translate a Guest Virtual Address (GVA)

- **VS-Stage**: Translates the Guest Virtual Address (GVA) to a Guest Physical Address (GPA) using the `vsatp` register (Sv39).
- **G-Stage**: Translates the Guest Physical Address (GPA) to the Host or Supervisor Physical Address (HPA or SPA). using the `hgap` register (Sv39x4).

Both `vsatp` and `hgap` support a *Bare* mode in addition to the respective Sv39 and Sv39x4 modes (similar to the `satp` register in non-virtualised mode)

- In `vsatp` bare mode $GVA = GPA$.
- In `hgap` bare mode $GPA = SPA$.

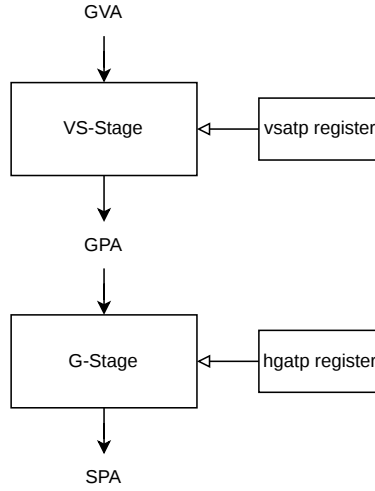


Figure 21: Two-step address translation scheme.

7.3 Hypervisor and Virtual Supervisor CSRs

Sargantana adds the standard set of [Hypervisor and Virtual Supervisor CSRs](#). These registers manage virtualization contexts, exception delegation, and two-stage translation.

Table 10: CSRs that were added for the hypervisor extension.

Register	Description
mtval2	Machine second trap Value (Faulting GPA right-shifted by 2 when a trap is taken into M-mode)
mtinst	Machine trap instruction (Faulting instruction or zero when a trap is taken into M-mode)
hstatus	Hypervisor status register, analogous to the <code>mstatus</code> register for tracking and controlling the exception behavior of a VS-mode guest
hedeleg / hideleg	Hypervisor exception and interrupt delegation registers.
hvip	Hypervisor virtual interrupt pending
hip / hie	Hypervisor interrupt pending/enable
hgeip / hgeie	Hypervisor guest external interrupt pending/enable.
henvcfg	Hypervisor environment configuration
hcounteren	Hypervisor counter enable
htimedelta	Hypervisor time delta for VS/VU mode
htval	Hypervisor trap value (Faulting GPA right-shifted by 2 when a trap is taken into HS-mode)
htinst	Hypervisor trap instruction (Faulting instruction or zero when a trap is taken into HS-mode)
hgatp	Hypervisor guest address-translation and protection, used instead of <code>satp</code> during G-Stage
vsstatus	Virtual supervisor status register, substitutes for <code>sstatus</code> when <code>v_mode = 1</code>

Table 10: CSRs that were added for the hypervisor extension.

Register	Description
<code>vsip</code> / <code>vsie</code>	Virtual supervisor interrupt pending/enable, substitutes for <code>sip</code> / <code>sie</code> when <code>v_mode = 1</code>
<code>vstval</code> / <code>vstvec</code>	Virtual supervisor trap value/vectore, substitutes for <code>stval</code> / <code>stvec</code> when <code>v_mode = 1</code>
<code>vsscratch</code>	Virtual supervisor scratch register, substitutes for <code>sscratch</code> when <code>v_mode = 1</code>
<code>vsatp</code>	Virtual supervisor address translation and protection, substitutes for <code>satp</code> when <code>v_mode = 1</code> and used during VS-Stage
<code>vsepc</code>	Virtual supervisor exception program counter register, substitutes for <code>sepc</code> when <code>v_mode = 1</code>
<code>vscause</code>	Virtual supervisor cause register, substitutes for <code>scause</code> when <code>v_mode = 1</code>

The `mstatus` register also includes the **MPV** (Machine Previous Virtualization Mode) and **GVA** (Guest Virtual Address) fields to accurately save and restore the virtualization state upon trapping to Machine mode.

7.4 Hypervisor Instructions

7.4.1 Hypervisor Virtual-Machine Load and Store Instructions (HLV / HSV)

Sargantana supports [hypervisor load and store instructions](#) (HLV / HSV). These instructions allow performing an explicit memory access from M-mode or HS-mode, or in U-mode when `hstatus.HU=1`, with an effective privilege mode of VS or VU. The effective privilege mode of the explicit memory access is VU when `hstatus.SPVP=0`, and VS when `hstatus.SPVP=1`.

For every load instruction, `LB`, `LBU`, `LH`, `LHU`, `LW`, `LWU`, and `LD`, there is a corresponding virtual-machine load instruction: `HLV.B`, `HLV.BU`, `HLV.H`, `HLV.HU`, `HLVX.HU`, `HLV.W`, `HLV.WU`, `HLVX.WU`, and `HLV.D`. For every store instruction: `SB`, `SH`, `SW`, and `SD`, there is a corresponding virtual-machine store instruction: `HSV.B`, `HSV.H`, `HSV.W`, and `HSV.D`.

Instructions `HLVX.HU` and `HLVX.WU` are the same as `HLV.HU` and `HLV.WU`, except that execute permission takes the place of read permission during address translation.

7.4.2 Hypervisor Memory-Management Fence Instructions

[HFENCE.VVMA](#) and [HFENCE.GVMA](#) instructions are supported to flush the virtualization-specific Translation Lookaside Buffer (TLB) entries. They perform a similar function to the `SFENCE.VMA` except that:

- `SFENCE.VMA` flushes the TLB entries of translations that used either the `satp` register when `v_mode = 0` or the `vsatp` register when `v_mode = 1`.
- `HFENCE.VVMA` flushes the TLB entries of translations that used the `vsatp` register and is valid only in M-mode or HS-mode.
- `HFENCE.GVMA` flushes the TLB entries of translations that used the `hgatp` register and is valid only in M-mode or in HS-mode when `mstatus.TVM = 0`.

7.5 Hypervisor Traps

Sargantana trap cause encoding is augmented by the hypervisor extension with [new hypervisor specific codes](#):

Table 11: Exception Causes introduced by the H-extension

Interrupt	Exception Code	Description
1	2	Virtual supervisor software interrupt
1	6	Virtual supervisor timer interrupt
1	10	Virtual supervisor external interrupt
1	12	Supervisor guest external interrupt
0	10	Virtual Supervisor ecall
0	20	Instruction guest page fault
0	21	Load guest page fault
0	22	Virtual instruction
0	23	Store/AMO guest page fault

When a guest-page-fault trap is taken into M-mode, `mtval2` is written with the guest physical address that faulted, shifted right by 2 bits. For other traps, `mtval2` is set to zero.

When a guest-page-fault trap is taken into HS-mode, `htval` is written with the guest physical address that faulted, shifted right by 2 bits. For other traps, `htval` is set to zero.

On any trap into M-mode or HS-mode, `mtinst` and `htinst` are written with zero.

8 Control/Status Registers (CSR)

The CSR's used by the core are defined in the [RISC-V Standard](#). Additionally, Sargantana adds custom CSRs with the address mapping shown in Table 12.

Table 12: Sargantana custom CSR mapping.

Region Name	Address
MEM_MAP0	0x7C0
MEM_MAP1	0x7C1
MEM_MAP2	0x7C2
MEM_MAP3	0x7C3
MEM_MAP4	0x7C4
MEM_MAP5	0x7C5
MEM_MAP6	0x7C6
MEM_MAP7	0x7C7
MEM_MAP8	0x7C8
MEM_MAP9	0x7C9
MEM_MAP10	0x7CA
MEM_MAP11	0x7CB
MEM_MAP12	0x7CC
MEM_MAP13	0x7CD
MEM_MAP14	0x7CE
MEM_MAP15	0x7CF
IO_MAP0	0x7D0
IO_MAP1	0x7D1
IO_MAP2	0x7D2

Table 12: Sargantana custom CSR mapping.

Region Name	Address
IO_MAP3	0x7D3
IO_MAP4	0x7D4
IO_MAP5	0x7D5
IO_MAP6	0x7D6
IO_MAP7	0x7D7
IO_MAP8	0x7D8
IO_MAP9	0x7D9
IO_MAP10	0x7DA
IO_MAP11	0x7DB
IO_MAP12	0x7DC
IO_MAP13	0x7DD
IO_MAP14	0x7DE
IO_MAP15	0x7DF
IRQ_MAP0	0x7E0
IRQ_MAP1	0x7E1
IRQ_MAP2	0x7E2
IRQ_MAP3	0x7E3
IRQ_MAP4	0x7E4
IRQ_MAP5	0x7E5
IRQ_MAP6	0x7E6
IRQ_MAP7	0x7E7
IRQ_MAP8	0x7E8
IRQ_MAP9	0x7E9
IRQ_MAP10	0x7EA
IRQ_MAP11	0x7EB
IRQ_MAP12	0x7EC
IRQ_MAP13	0x7ED
IRQ_MAP14	0x7EE
IRQ_MAP15	0x7EF
-	-
HYPERRAM_CONFIG	0x7F0
CNM_CONFIG	0x7F1
SPI_CONFIG	0x7F2
-	-
FTRAN	0x800
DYN_MEM_LATENCY	0x801
-	-
TO_HOST	0x9F0
FROM_HOST	0x9F1
CLEAR_MIP	0x9F2
-	-
MBOOT_MAIN_ID	0xFC0

9 Hardware Performance Monitor (HPM)

In Sargantana we have support for the following extensions related with HPM counters (note that the Zicntr and Zihpm extensions were previously part of the base ISA):

- Zicntr: Adds the CYCLE, TIME and INSTRET counters (for cycle count, real-time clock and

instructions retired respectively).

- Zihpm: Adds up to 29 additional unprivileged 64-bit hardware performance counters, `hpmcounter3`–`hpmcounter31` (the set of events they count is platform specific). The event selector CSRs, `mhpmevent3`–`mhpmevent31`, are 64-bit registers that control which event causes the corresponding counter to increment.
- Sscofpmf: Adds privilege mode filtering to disable counting events based on the current privilege and a new interrupt to detect the overflow of counters.

More information can be found in the [RISC-V Standard](#).

Table 13: Sargantana HPM events.

HPM Event Name	ID
Branch Misses	1
Executed Branches	2
Taken Branches	3
Executed Stores	4
Executed Loads	5
iCache Requests	6
iCache Kills	7
Fetch Stage Stalls	8
Decode Stage Stalls	9
Register Read Stage Stalls	10
Execution Stage Stalls	11
Writeback Stage Stalls	12
iCache Fill from L2	13
iCache Killed	14
iCache Busy	15
iCache Miss Cycles	16
Cycles of Load Blocked by Store	17
Stalls by Data Dependencies	18
Stalls by Structural Risks	19
Stalls by Graduation List Full	20
Stalls by Free List Empty	21
iTLB access	22
iTLB miss	23
dTLB access	24
dTLB miss	25
PTW cache hit	26
PTW cache miss	27
Stalls by iTLB miss	28
dCache stalls	29
dCache refill stalls	30
dCache rtab rollback	31
dCache request on hold	32
dCache prefetch request	33
dCache read request	34
dCache write request	35
dCache CMO request	36
dCache uncached request	37
dCache read request miss	38
dCache write request miss	39
Instruction Rename Stage Stalls	40

Table 13: Sargantana HPM events.

HPM Event Name	ID
L3 Miss	41
L3 Access	42
L2 Miss	43
L2 Access	44
NoC 1 Flit	45
NoC 2 Flit	46
NoC 3 Flit	47
NoC 1 Stall	48
NoC 2 Stall	49
NoC 3 Stall	50

10 Debug Mode Extension (Sdext)

In Sargantana we have support for a minimal subset of RISC-V’s Debug Mode Extension (Sdext). This section contains the specification of the implemented portion of the RISC-V debug extension, Sdext, version 1.0.0-rc3. The complete documentation of this extension can be found in the following [URL](#). In short, this section focuses on the implementation of the ISA extension only, the implemented CSRs, and the behavior of the core under the Debug Mode. More specifically, the detailed behavior explained are the Debug Mode itself, lr/sc atomic operations, single stepping and the “Reset”, “Halt” and “Resume” states.

10.1 Debug Mode

When executing code due to an abstract command, the hart stays in Debug Mode and the following apply:

1. All implemented instructions operate just as they do in M-mode, unless an exception is mentioned in this list.
2. All operations are executed with machine mode privilege, except that additional Debug Mode CSRs are accessible and mprv in mstatus may be ignored according to mprven. Full permission checks, or a relaxed set of permission checks, will apply according to relaxedpriv.
3. All interrupts (including NMI) are masked.
4. Traps don’t take place. Instead, they end execution of the program buffer and the hart remains in Debug Mode. Because they do not trap to M-mode, they do not update registers such as , mepc, mcause, mtval, mtval2, and mtinst. The same is true for the equivalent privileged registers that are updated when trapping to other modes. Registers that may be updated as part of execution before the exception are allowed to be updated. For example, vector load/store instructions which raise exceptions may partially update the destination register and set vstart appropriately.
5. If stopcount is 0 then counters continue. If it is 1 then counters are stopped.
6. If stoptime is 0 then time continues to update. If it is 1 then time will not update. It will resynchronize with time after leaving Debug Mode.
7. Instructions that place the hart into a stalled state act as a nop. This includes wfi, wrs.sto, and wrs.nto.
8. Almost all instructions that change the privilege mode act as nop. This includes ecall, mret, sret, and uret. (To change the privilege mode, the debugger can write prv and v in dcsr). The only exception is ebreak, which ends execution of the Program Buffer when executed.

9. All control transfer instructions act as illegal instructions if their destination is in the Program Buffer. If one such instruction acts as an illegal instruction, all such instructions must act as illegal instructions.
10. All control transfer instructions act as illegal instructions if their destination is outside the Program Buffer. If one such instruction acts as an illegal instruction, all such instructions must act as illegal instructions.
11. Instructions that depend on the value of the PC (e.g. auipc) act as illegal instructions.
12. Effective XLEN is $DXLEN = 64$ bits.
13. Forward progress is guaranteed.

10.2 Load-Reserved/Store-Conditional Instructions

Hardware implementation does not care or prevent the preservation of the lr instruction.

10.3 Wait for Interrupt Instruction

If halt is requested while wfi is executing, then the hart leaves the stalled state, completing this instruction's execution, and then enters Debug Mode.

10.4 Wait-on-Reservation-Set Instructions

Sargantana does not support these instructions.

10.5 Single Step

Single stepping is implemented using the “step” bit in the Dcsr, as defined in the Sdext specification section 4.5.1.

10.6 Reset

If the halt signal (driven by the hart's halt request bit in the Debug Module) or hasresethaltreq are asserted when a hart comes out of reset, the hart must enter Debug Mode before executing any instructions, but after performing any initialization that would usually happen before the first instruction is executed.

10.7 Halt

When a hart halts:

1. cause is updated.
2. prv are set to reflect current privilege mode.
3. dpc is set to the next instruction that should be executed.
4. The halt will take place after all the in-flight instructions are committed.
5. The hart enters Debug Mode.

10.8 Resume

When a hart resumes:

1. pc changes to the value stored in dpc.
2. The current privilege mode are changed to that specified by prv.
3. If the new privilege mode is less privileged than M-mode, MPRV in mstatus is cleared.
4. The hart exits debug mode.

10.9 Core Debug Registers

Attempts to access an unimplemented Core Debug Register raise an illegal instruction exception. These registers are only accessible from Debug Mode.

Table 14: Debug Registers.

Address	Name	Section
0x7b0	Debug Control and Status (dcsr)	Section 10.9.1
0x7b1	Debug PC (dpc)	Section 10.9.2
0x7b2	Debug Scratch Register 0 (dscratch0)	Section 10.9.3
0x7b3	Debug Scratch Register 1 (dscratch1)	Section 10.9.4

10.9.1 Debug Control and Status (dcsr, at 0x7b0)

Upon entry into Debug Mode, `prv` is updated with the privilege level the hart was previously in, and `cause` is updated with the reason for Debug Mode entry. Other than these fields and `nmip`, the other fields of `dcsr` are only writable by the external debugger.

Table shows the priorities of reasons for entering Debug Mode:

Table 15: Encoding and priorities of the causes for entering Debug Mode.

cause encoding	Cause
5	resethaltreq
6	halt group
3	haltreq
2	trigger (Not implemented)
1	ebreak
4	step

This CSR is read/write.

Table 16: Encoding and priorities of the causes for entering Debug Mode.

Field	Description	Access	Reset
debugver	Hardwired to 4 (Debug support exists as it is described in 1.0 specs)	R	Preset
extcause	Hardwired to 0. (not implemented)	R	Preset
cetrig	Hardwired to 0. (not implemented)	R/W	Preset
ebreakvs	This bit is hardwired to 0, hart does not support virtualization mode.	WARL	Preset
ebreakvu	This bit is hardwired to 0, hart does not support virtualization mode.	WARL	Preset
ebreakm	0 (exception): ebreak instructions in M-mode behave as described in the Privileged Spec. 1 (debug mode): ebreak instructions in M-mode enter Debug Mode.	R/W	0
ebreaks	0 (exception): ebreak instructions in S-mode behave as described in the Privileged Spec. 1 (debug mode): ebreak instructions in S-mode enter Debug Mode	WARL	0

Table 16: Encoding and priorities of the causes for entering Debug Mode.

Field	Description	Access	Reset
ebreaku	0 (exception): ebreak instructions in U-mode behave as described in the Privileged Spec. 1 (debug mode): ebreak instructions in U-mode enter Debug Mode	WARL	0
stepie	Hardwired to 0: Interrupts (including NMI) are disabled during single stepping with step set. Interrupt behavior is emulated by the debugger.	WARL	Preset
stopcount	Hardwired to 0: Increment counters as usual.	WARL	Preset
stoptime	Hardwired to 0: Increment time as usual.	WARL	Preset
cause	Explains why Debug Mode was entered (depending on the priority of each cause). 1 (ebreak): An ebreak instruction was executed. 2 (trigger): Not implemented. 3 (haltreq): The debugger requested entry to Debug Mode using haltreq. 4 (step): The hart single stepped because step was set. 5 (resethaltreq): The hart halted directly out of reset due to resethaltreq. It is also acceptable to report 3 when this happens. 6 (group): The hart halted because it's part of a halt group. Harts may report 3 for this cause instead. 7 (other): Not implemented.	R	0
v	Extends the prv field with the virtualization mode the hart was operating in when Debug Mode was entered. A debugger can change this value to change the hart's virtualization mode when exiting Debug Mode.	WARL	0
mprven	Hardwired to 0: mprv in mstatus is ignored in Debug Mode	WARL	Preset
nmip	When set, there is a Non-Maskable-Interrupt (NMI) pending for the hart.	R	0
step	When set and not in Debug Mode, the hart will only execute a single instruction and then enter Debug Mode.	R/W	0
prv	Contains the privilege mode the hart was operating in when Debug Mode was entered. A debugger can change this value to change the hart's privilege mode when exiting Debug Mode. 0 - U-mode 1 - S-mode 3 - M-mode	WARL	3

10.9.2 Debug PC (dpc, at 0x7b1)

Upon entry to debug mode, dpc is updated with the virtual address of the next instruction to be executed. The behavior is described in more detail in the following table.

Table 17: Debug Registers.

Cause	Virtual Address in DPC
ebreak	Address of the ebreak instruction
Single step	Address of the instruction that would be executed next if no debugging was going on. I.e. $pc + 4$ for 32-bit instructions that don't change program flow, the destination PC on taken jumps/branches, etc.
Trigger module	Not implemented
Halt request	Address of the next instruction to be executed at the time that debug mode was entered.

When resuming, the hart's PC is updated to the virtual address stored in dpc. A debugger may write dpc to change where the hart resumes.

This CSR is read/write.

10.9.3 Debug Scratch Register 0 (dscratch0, at 0x7b2)

Scratch register that can be used by implementations that need it.

This CSR is read/write.

10.9.4 Debug Scratch Register 1 (dscratch1, at 0x7b3)

Scratch register that can be used by implementations that need it.

This CSR is read/write.

10.10 Virtual Debug Registers

Sargantana does not provide any hardware implementation of the virtual registers.

11 Future Work

At the time of writing, the main goal of the Sargantana team is to continue the development of the core to have support for the RISC-V [RVA23 profile](#). The current missing extensions are:

- "C" Compressed Instructions
- "Zicclsm" Main memory supports misaligned loads/stores
- "Zimop" May-Be-Operations
- "Zcmop" Compressed May-Be-Operations
- "Zcb" Simple compressed instructions
- "Zawrs" Wait-on-Reservation-Set Instructions
- "Supm" & "Ssnpm" Pointer masking support
- "Svpbmt" Page-Based Memory Types
- "Svinval" Fine-Grained Address-Translation Cache Invalidation
- "Svnapot" NAPOT Translation Contiguity
- "Sstc" Supervisor-mode Timer Interrupts

References

- [1] Victor Soria et al. "Sargantana: A 1 GHz+ in-order RISC-V processor with SIMD vector extensions in 22nm FD-SOI". In: *2022 25th Euromicro Conference on Digital System Design (DSD)*. IEEE. 2022, pp. 254–261.
- [2] César Fuguet. "HPDcache: Open-Source High-Performance L1 Data Cache for RISC-V Cores". In: *Proceedings of the 20th ACM International Conference on Computing Frontiers*. CF '23. Bologna, Italy: Association for Computing Machinery, 2023, pp. 377–378. ISBN: 9798400701405. DOI: [10.1145/3587135.3591413](https://doi.org/10.1145/3587135.3591413). URL: <https://doi.org/10.1145/3587135.3591413>.
- [3] Stefan Mach et al. "Fpnew: An open-source multiformat floating-point unit architecture for energy-proportional transprecision computing". In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 29.4 (2020), pp. 774–787.
- [4] Nebojša Z. Milenković, Vladimir V. Stanković, and Miljana Lj. Milić. "Modular Design Of Fast Leading Zeros Counting Circuit". In: *Journal of Electrical Engineering* 66.6 (2015), pp. 329–333. DOI: [10.2478/jee-2015-0054](https://doi.org/10.2478/jee-2015-0054). URL: <https://doi.org/10.2478/jee-2015-0054>.